Proving the Exact Minimum CNOT Length of a 13×13 Binary Transformation

Ivan Litvak
Independent Researcher
vanya.litvak@gmail.com

27 September 2026

Abstract

We establish that the minimum CNOT length of the explicit binary matrix below is nineteen. Structural lower bounds from projected XOR complexity, forced copies and restorations, and clean-carrier invariants reduce the universal lower bound to 363 finite cases. Exact passive-target elimination then permits an exhaustive external C++ check; a nineteen-gate witness establishes attainability. Lean formalizes the general reduction and proves the minimum conditional on two precisely stated finite facts about the concrete lookup tables and search. We include the verifier, complete-run evidence, and Lean sources. The finite facts are supported by an audited external computation, not replayed by the Lean kernel.

1 The matrix and the exact-minimum statement

Let $\mathbb{F}_2 = \{0, 1\}$ with addition modulo two. The input is an arbitrary vector $x = (x_0, \ldots, x_{12})^\top \in \mathbb{F}_2^{13}$; its thirteen coordinates are independent variables, not values restricted to a code space. We study the full labelled linear map $x \mapsto Mx$. With both rows and columns ordered $0, 1, \ldots, 12$, the matrix is explicitly

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}. \tag{1}$$

Definition 1 (Allowed programs and their length)**.** *A current wire value X_i is represented by its linear form in the thirteen inputs; initially $X_i = x_i$, so the matrix of current forms is I_{13}. A directed CNOT gate (c, d), with control c, target d, and $c \neq d$, replaces X_d by $X_d + X_c$ and leaves every other row unchanged. Thus every ordered pair of distinct labelled coordinates is allowed: there are exactly $13 \cdot 12 = 156$ gates, each of unit cost. A chronological list $g_1, \ldots, g_L$ applies these row*

updates successively to I_{13}; its cost is L. We use program *and* word *interchangeably for such a chronological gate list. We define*

$$\ell(M) = \min\{L : \textit{a legal length-}L\textit{ program maps } I_{13} \textit{ to } M\}.$$

Equality with M means equality of all thirteen output linear forms for all $x \in \mathbb{F}_2^{13}$. There are no prepared zero inputs, additional coordinates, Hadamard gates, measurements, or free input/output permutations. In particular, the coordinate order is fixed throughout. We minimize the cost of the full labelled operator, not its action on an encoding subspace.

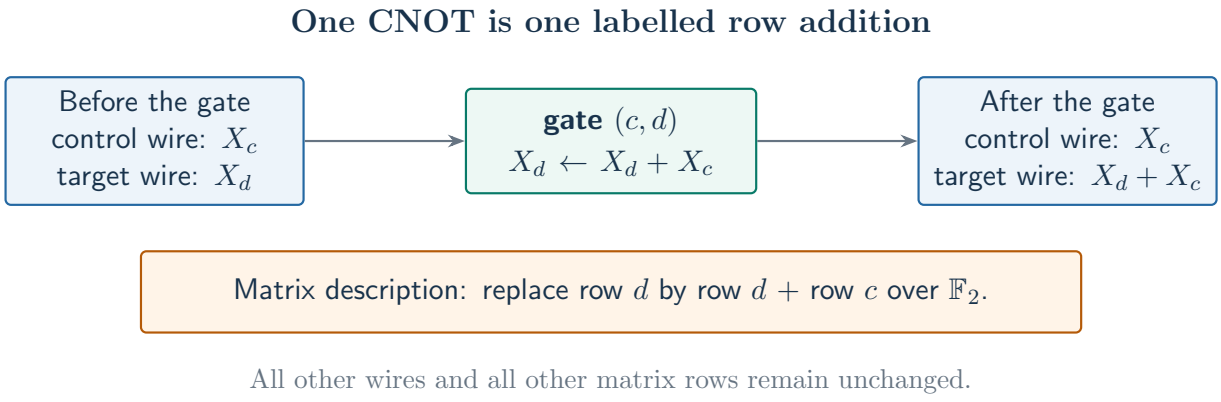


Figure 1. One allowed gate in all three equivalent descriptions. The physical statement $X_d \leftarrow X_d + X_c$ is the same as adding row c to row d over $\mathbb{F}_2$. Only the target changes, so a length-L word performs exactly L row additions.

Theorem 2. *The matrix in* (1) *has exact minimum length $\ell(M) = 19$. Equivalently, every legal program implementing M has at least nineteen gates, and a legal nineteen-gate program exists.*

The proof is organized around the universal lower bound. The finite exclusion needed after the structural reduction is reproducible from the included source. Its rerun checks the same implementation, not a separately implemented solver. Section 11 states the precise current Lean-verification boundary.

A row r is encoded by $\sum_{j=0}^{12} r_j 2^j$. This fixes every matrix entry and the bit order. For comparison with the machine-readable data, the same matrix has row masks

$$(M_0, \ldots, M_{12}) = (1, 2, 4, 8, 16, 32, 329, 402, 292, 515, 1030, 2075, 4150). \tag{2}$$

Write $S = \{0, 1, 2, 3, 4, 5\}$, let coordinate 8 be the *central* coordinate, and order the remaining coordinates as $Y = (6, 7, 9, 10, 11, 12)$. Use input names (a, b, c, d, e, f) on S, t on coordinate 8, and $(y_0, \ldots, y_5)$ on Y. The operator is

$$\begin{aligned}
(a', b', c', d', e', f') &= (a, b, c, d, e, f), & t' &= t + c + f,\\
y_0' &= y_0 + a + d + t, & y_1' &= y_1 + b + e + t,\\
y_2' &= y_2 + a + b, & y_3' &= y_3 + b + c,\\
y_4' &= y_4 + a + b + d + e, & y_5' &= y_5 + b + c + e + f.
\end{aligned}$$

The coordinates in S will be called *sources*: their final rows are unchanged basis rows. The coordinates in Y will be called *private outputs*; "private" refers only to the displayed incidence pattern—each row retains its own independent input y_j—and never means that y_j is zero or initialized specially. Let

$$J = S \cup \{8\}, \qquad \pi : \mathbb{F}_2^{13} \longrightarrow \mathbb{F}_2^7$$

be the coordinate projection that retains the coefficients on J, in the order $(0, 1, 2, 3, 4, 5, 8)$, and deletes the six coefficients on Y. Thus an initial private row has projection zero even though its physical input y_j is arbitrary.

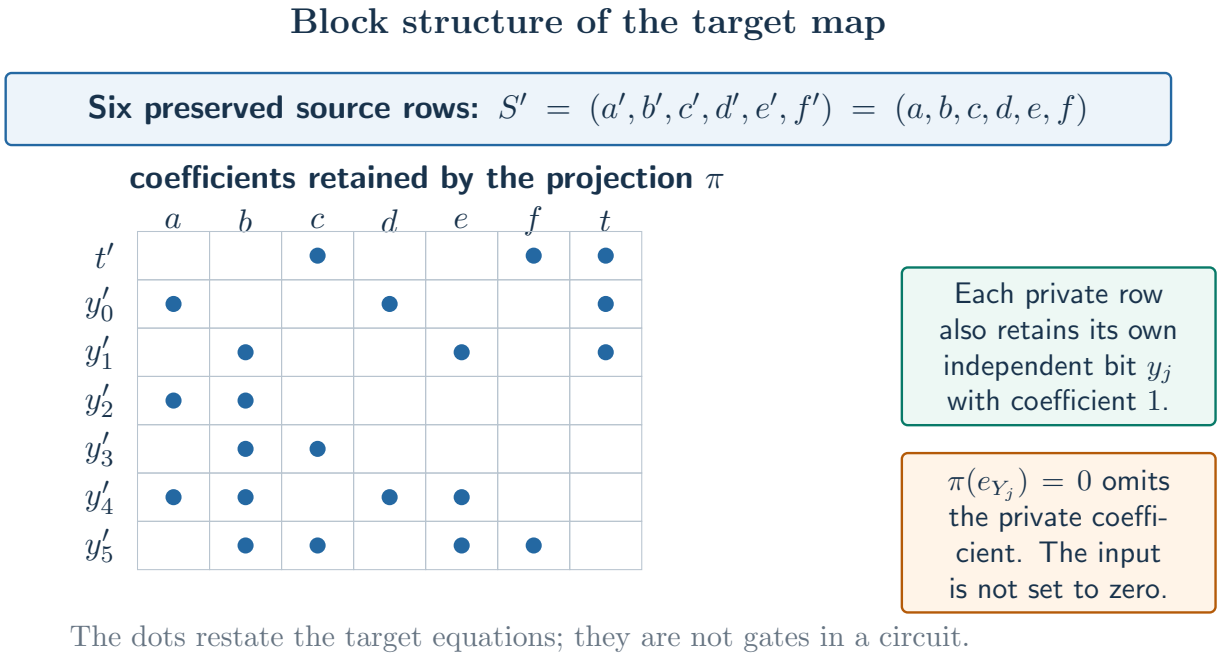


Figure 2. Block structure of the target transformation. A dot marks a retained coefficient equal to one. Every private output also retains its own independent input y_j; the projection π merely omits that coefficient in the auxiliary seven-bit problem. The picture restates the target equations and is not a circuit.

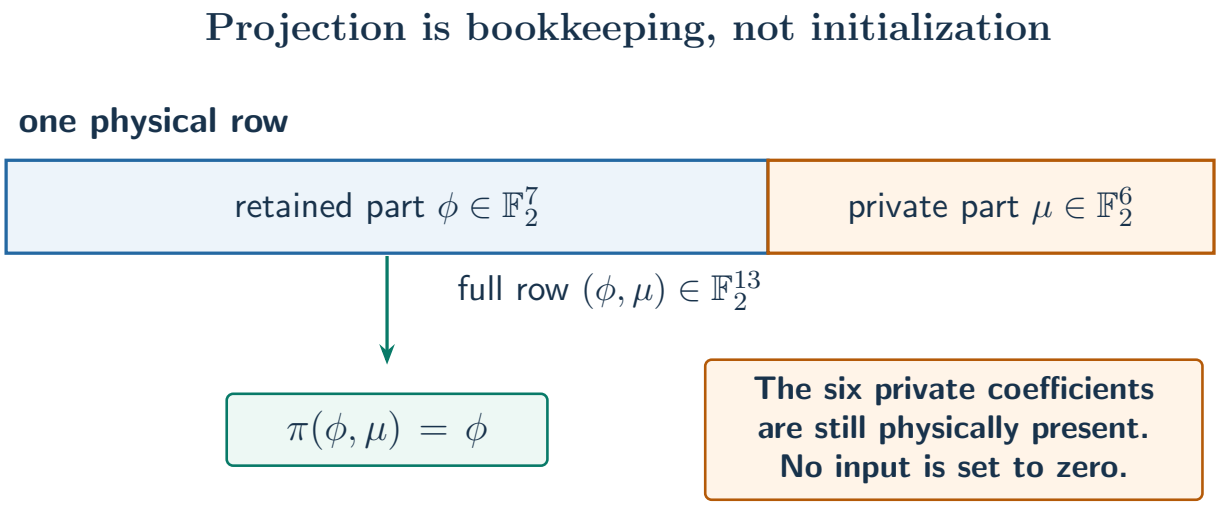


Figure 3. The two views of one physical row. Its full thirteen-bit value consists of seven retained coefficients and six private-input coefficients. The projection π keeps the first block and forgets the second only for bookkeeping; it does not set any physical input to zero and does not modify the circuit.

For a program $\mathcal{P}$, define

$$u(\mathcal{P}) = |\{s \in S : \text{some gate of } \mathcal{P} \text{ targets } s\}|,$$
$$v(\mathcal{P}) = |\{y \in Y : \text{some gate of } \mathcal{P} \text{ uses } y \text{ as control}\}|.$$

These count distinct labelled coordinates, not gate occurrences. We write u and v when the program is understood.

1.1 Notation guide

The proof uses several representations of the same circuit. The following tables are a lookup guide; every local symbol is also defined again where it first enters an argument.

Symbol	Meaning
x	arbitrary physical input vector in $\mathbb{F}_2^{13}$
X_i	current linear form stored on labelled wire i
M	the fixed target matrix in (1)
L	length of the particular program currently under discussion
$\ell(M)$	minimum legal CNOT length of the target matrix
S	the six source coordinates $\{0, \dots, 5\}$
t	the central input, physically located on coordinate 8
Y	the six private-output coordinates $(6, 7, 9, 10, 11, 12)$
J	the seven retained coordinates $S \cup \{8\}$
π	projection of a full row onto the coefficients indexed by J
u	number of distinct sources ever used as targets
v	number of distinct private outputs ever used as controls

Later sections use P for the seven initial projected basis forms, $f_0, \dots, f_6$ for the seven required projected increments, F for their ordered list, and α for free XOR complexity. The counting variables h and ρ denote, respectively, the number of additional projected forms and the number of unclassified repeated projected results. These names are chosen to avoid confusion with the physical source variables (a, b, c, d, e, f).

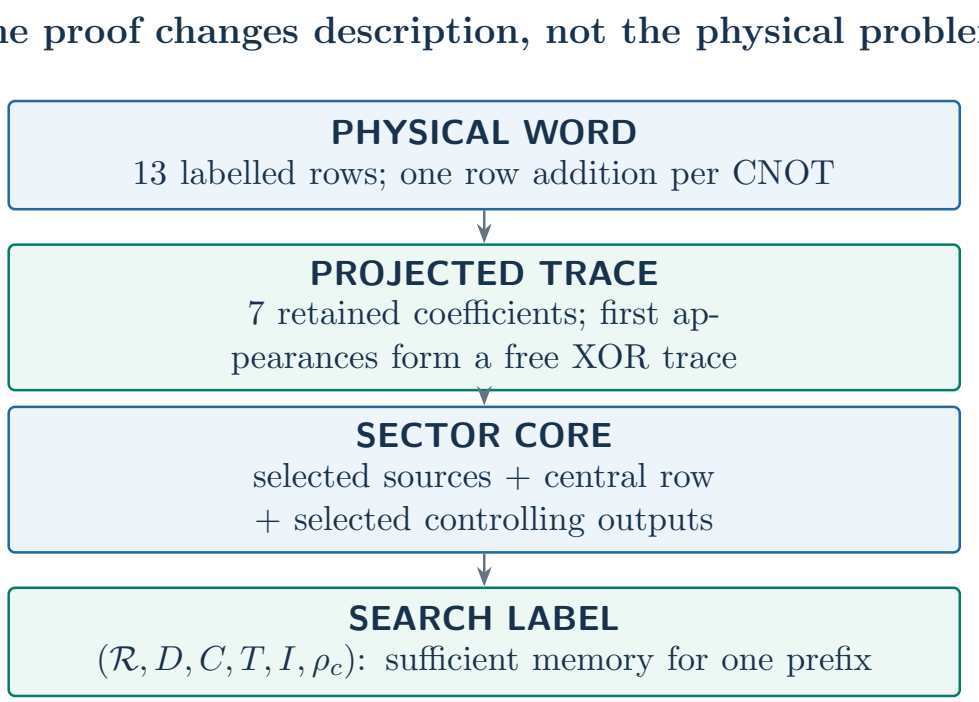


Figure 4. The four levels of description used in the proof. The physical word acts on thirteen full rows. Projection keeps seven coefficients. A sector selects a smaller core plus passive outputs. The finite search then stores only a sufficient label for each core prefix. Every arrow is justified later; none changes the physical synthesis problem.

1.2 How the lower bound works

Figure 5 separates the structural deductions from the remaining finite exclusion. The proof has five stages:

1. the explicit witness gives the upper bound $\ell(M) \leq 19$;
2. projection by π, followed by forced-copy and restoration counts, gives the two budgets $L \geq 16 + u$ and $L \geq 16 + v$;
3. these budgets and the separate $u = v = 0$ argument place every hypothetical program of length at most eighteen in one of 363 sectors;
4. catalogue, clean-carrier, passive-target, and remaining-cost tests define a finite search in each sector; and
5. the C++ run finds no terminal state of total cost at most eighteen.

Lean formalizes the general reduction and abstract search; the concrete finite exclusion remains an external computation, as detailed in Section 11.

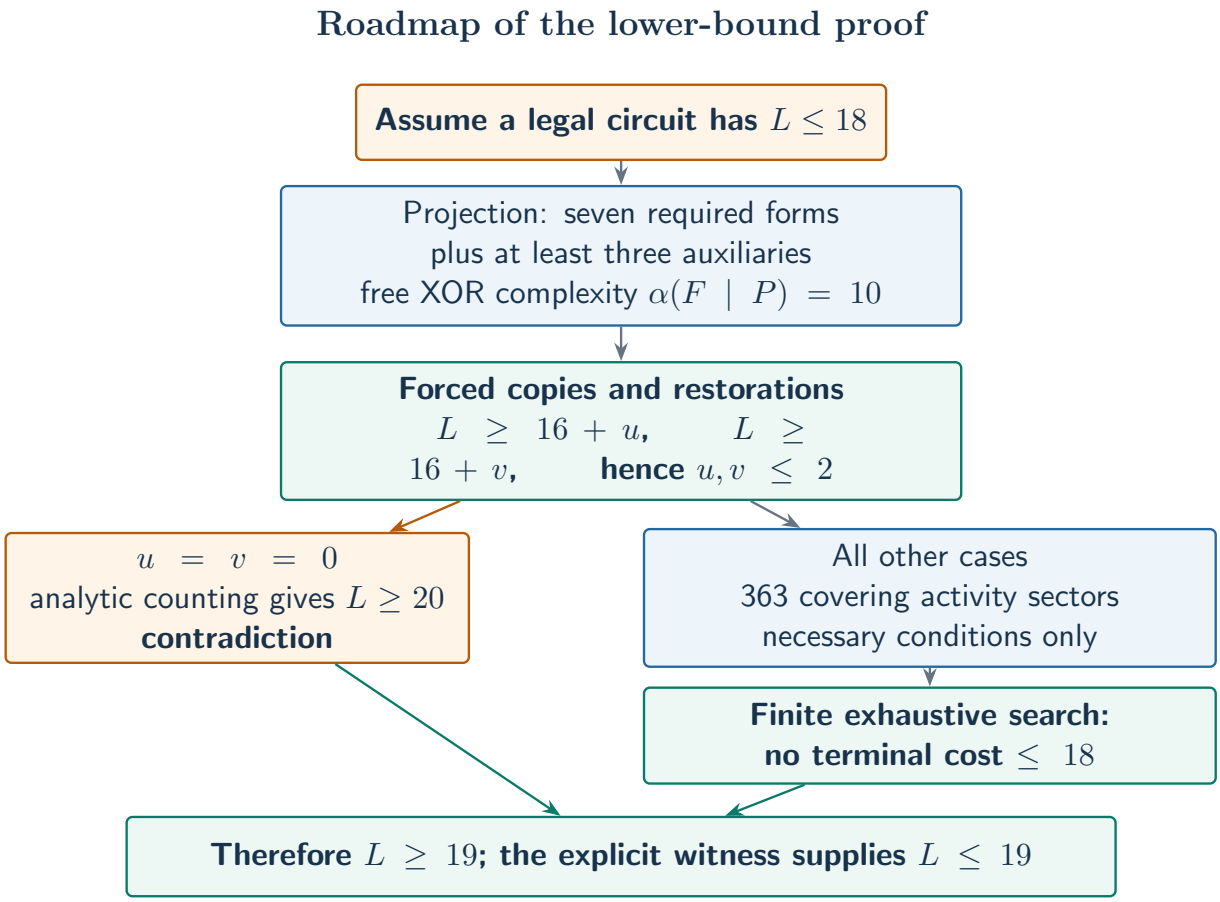


Figure 5. Logical structure of the lower bound. Every program of length at most eighteen must follow one of the displayed branches; the mathematical reductions plus the reported exhaustive search rule out every branch. The diagram is a roadmap, not a search certificate or a claim that every displayed implication has been checked in Lean. The nineteen-gate upper bound is separate.

2 Related results and provenance

Patel, Markov, and Hayes prove the asymptotic worst-case CNOT count $\Theta(n^2/\log n)$ [1]. This does not determine the cost of (2) or an exact diameter at $n = 13$. Bu, Fan, and Joo give a general lower bound and prove that an n-cycle permutation requires $3(n-1)$ CNOT gates [2]. In particular, a 13-cycle requires 36 gates, so the present nineteen-gate matrix is not a diametral example. Their computable LMC estimate evaluates to 12 for our matrix and for its transpose, inverse, and inverse transpose; it does not directly establish the claimed value 19.

Sodhani and Parhi report a nineteen-gate circuit for an HGP $[[13, 1]]$ encoder [3]. Their benchmark table supplies an achieved count, not an exclusion of all eighteen-gate realizations. Our local reconstruction uses a length-three repetition-code parity-check matrix and fixed elimination conventions. It is not author-supplied benchmark data. Theorem 2 concerns only (2); no stronger benchmark-identity or priority claim is made. Standard XOR-DAG and transposition arguments used below are not claimed as new general methods.

3 An explicit upper bound

Figure 6 shows the witness in the original coordinate order. It establishes $\ell(M) \le 19$; the later sections establish the central claim $\ell(M) \ge 19$. The following chronological program realizes M:

$$(0,9),(1,9),(1,10),(2,10),(2,12),(5,12),$$
$$(3,0),(0,11),(4,1),(1,11),(1,12),(1,7),(0,6),$$
$$(8,7),(8,6),(2,8),(5,8),(4,1),(3,0).$$

The successive new target values are

$$a+y_2,\ a+b+y_2,\ b+y_3,\ b+c+y_3,\ c+y_5,\ c+f+y_5,$$
$$a+d,\ a+d+y_4,\ b+e,\ a+b+d+e+y_4,$$
$$b+c+e+f+y_5,\ b+e+y_1,\ a+d+y_0,$$
$$b+e+t+y_1,\ a+d+t+y_0,\ c+t,\ c+f+t,\ b,\ a.$$

These give precisely (2). Each gate is an involution, so the target is invertible as well.

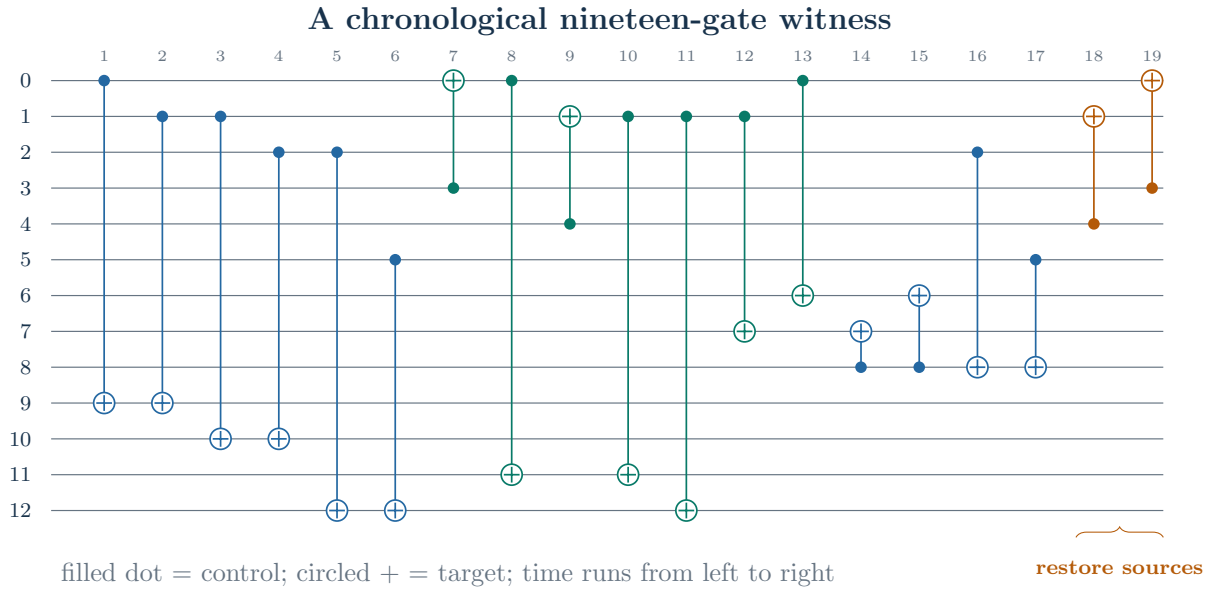


Figure 6. The chronological nineteen-gate witness. The two final gates restore the temporarily changed source rows. This diagram is generated from the same gate list as the Lean witness and checked against all thirteen rows of M. It proves only the upper bound $\ell(M) \le 19$; it does not address optimality.

4 Projected XOR complexity

Apply π to each current row, discarding its six private-input coefficients only in this auxiliary argument. No physical input is removed or fixed: the original thirteen inputs remain arbitrary. The projected problem is deliberately easier than the physical one: it remembers every form once produced, permits unlimited reuse, and never overwrites a form. Therefore the first appearances in any physical CNOT program form a valid free XOR construction. The initial nonzero projected forms are $P = \{a, b, c, d, e, f, t\}$. The seven required new forms are

$$f_0 = r_0 = a+d+t, \qquad f_1 = r_1 = b+e+t, \qquad f_6 = r_2 = c+f+t,$$
$$f_2 = q_0 = a+b, \qquad f_3 = q_1 = b+c, \qquad f_4 = s_0 = r_0 + r_1, \qquad f_5 = s_1 = r_1 + r_2.$$

Thus f_j is the required projected increment of private output y_j for $0 \le j < 6$, while f_6 is the required final projected value of the central row. In the order $(f_0, \dots, f_6)$ their bit masks are

$$F = (73, 82, 3, 6, 27, 54, 100). \tag{3}$$

A free XOR program retains all previous forms and adds two available forms at unit cost. Write $\alpha(F \mid P)$ for its minimum cost.

Lemma 3. $\alpha(F \mid P) = 10$.

Proof. Let $H = \operatorname{span}(r_0, r_1, r_2)$. The coefficients of d, e, f give $\dim H = 3$, and its masks are $\{0, 27, 45, 54, 73, 82, 100, 127\}$. In $\mathbb{F}_2^7/H$, the classes of a, b, c, t are independent, with $d \equiv a + t$, $e \equiv b + t$, and $f \equiv c + t$. The nine elements of $(P \cup F) \setminus H$ therefore have distinct nonzero classes, and $P \cap H = \varnothing$.

The intuition is that a new form outside H can create at most one new independent difference inside its coset, whereas the targets inside H require three independent directions. We now make that count precise.

Let B_{aux} be the distinct additional computed forms outside H and outside $P \cup F$. Let $W \subseteq H$ be spanned by differences of pairs in $((P \cup F) \setminus H) \cup B_{\text{aux}}$ belonging to the same coset. Every computed form inside H belongs to W: induct on the program. Two parents inside H stay in W; exactly one parent in H cannot produce an element of H; two parents outside H whose sum is in H give one of the defining differences. Thus $\dim W \ge 3$. Initially there are no such differences. Adding one element to a coset increases the dimension of their span by at most one, because after choosing one representative all new differences are generated by one new difference and the old ones. Hence $|B_{\text{aux}}| \ge 3$. The seven distinct target forms and at least three additional forms require at least ten additions. Conversely, compute $a + d, b + e, c + f$, then their sums with t, the two q forms, and the two adjacent pair sums of these three auxiliary forms. This uses ten additions. □

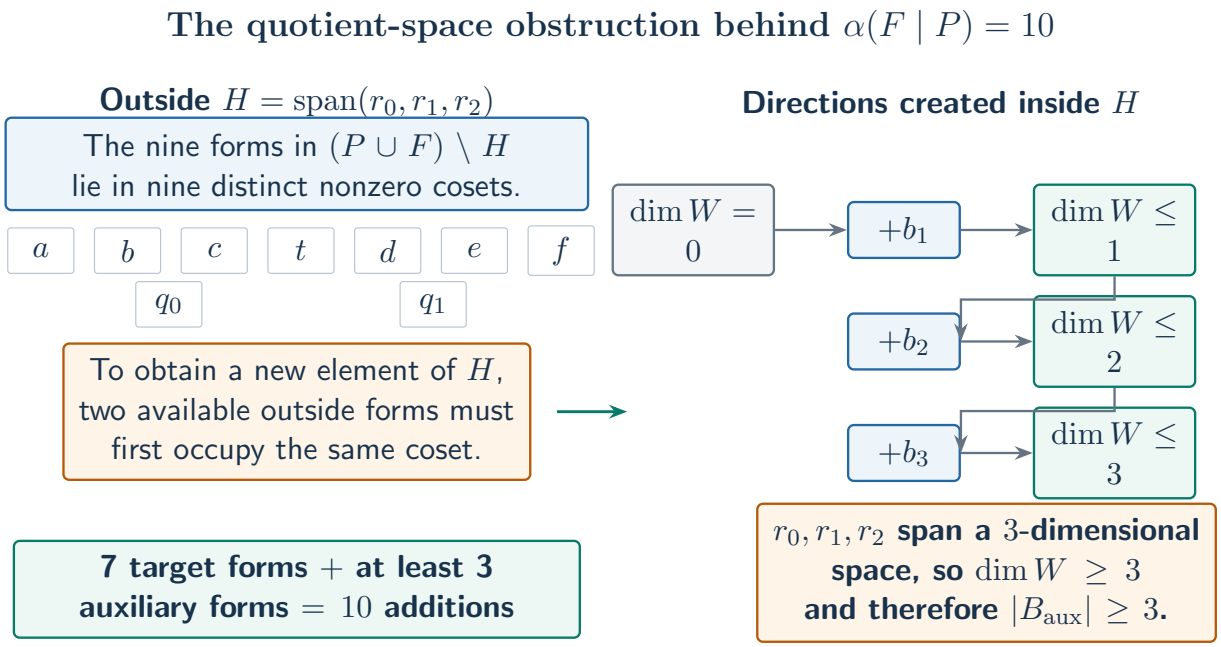


Figure 7. The dimension obstruction in Lemma 3. Equalities such as $d = a + t$ are identities of cosets modulo H. The nine outside forms occupy distinct nonzero cosets; each additional outside form contributes at most one new direction to W. This proves the projected free-XOR lower bound, not by itself a lower bound for full thirteen-wire CNOT programs.

For a binary matrix T whose columns represent independent input forms, write $\alpha(T)$ for its free XOR complexity from the basis inputs. The next lemma says that an XOR computation graph can be read backwards.

Lemma 4. *For a binary $m \times n$ matrix T without zero rows or columns, free XOR complexity satisfies $\alpha(T^\top) = \alpha(T) + m - n$.*

Proof. Here n is the number of input forms and m the number of requested output forms. Remove unused computation nodes from a circuit with s additions, leaving $s' \le s$, and attach one pointer to each of the m outputs. Every remaining input or computation node has at least one outgoing use. The total number of uses, including multiple edges and output pointers, is $2s' + m$. Reverse the edges. If a node v had d_v outgoing uses, it now has d_v incoming values and sums them using $d_v - 1$ additions. The total is $\sum_v (d_v - 1) = s' + m - n$. The parity of directed paths describes every input-output coefficient; reversing edges transposes these coefficients. This proves one inequality; applying it to $T^\top$ proves the reverse. □

The matrix with rows (3) is square without zero rows or columns; its transpose has rows

$$F^\top = (21, 62, 104, 17, 50, 96, 67). \tag{4}$$

Its free complexity is also ten. Transposing a full CNOT program reverses its order and exchanges each control and target, preserving length. The normalized coordinate orders used below are $(0, 1, 2, 3, 4, 5, 8, 6, 7, 9, 10, 11, 12)$ and $(6, 7, 9, 10, 11, 12, 8, 0, 1, 2, 3, 4, 5)$ respectively. This is a length-preserving correspondence between two synthesis problems, not a free operation available inside the original gate model.

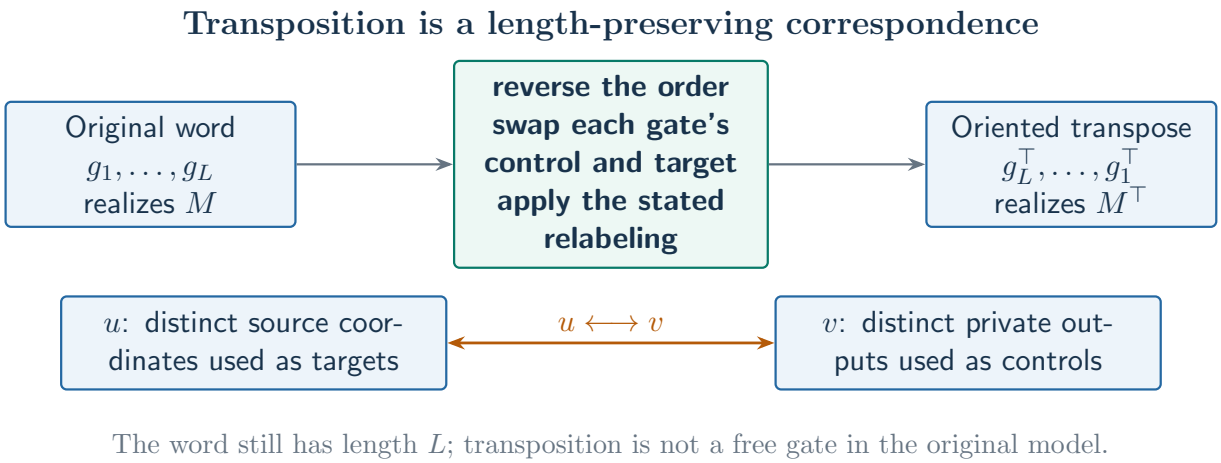


Figure 8. How the transpose reduction acts on a full CNOT word. Reversing the word and swapping every control-target pair preserves its length and exchanges the activity statistics u and v after the stated relabeling. It does not authorize a free transpose or permutation within the original synthesis problem.

5 A budget for repeated forms

Fix a full program and use the program statistics u and v defined above. Every gate is classified by the projected value of its new target row. A *first appearance* creates a projected form not seen earlier. A *repeat* produces a projected form already seen. We write h for the number of additional first-appearing forms outside $P \cup F \cup \{0\}$, and ρ for repeated results that are neither a forced first copy nor a credited source restoration.

Lemma 5. *Every length-L realization satisfies $L \geq 16+u$ and $L \geq 16+v$. More precisely, in one orientation,*

$$L = 13 + h + u + \rho, \qquad h \geq 3, \quad \rho \geq 0. \tag{5}$$

Proof. Retain all previously observed projected forms, including initial zero. First occurrences give a free XOR program with $7 + h \geq 10$ additions. Each of the six private outputs starts with zero projection and ends nonzero; its first nonzero update copies an already available projected control, and is a repeat. Each changed source must finally restore its original full row, producing another repeat. These $6+u$ events are distinct. The remaining ρ repeated events give (5). Apply the same argument to the transposed word to obtain the bound in v. □

Thus $L \leq 18$ implies $u, v \leq 2$. If $u = 2$, then $L = 18, h = 3, \rho = 0$. If $u = 1$, then $h + \rho \leq 4$. The two lower bounds are never added together.

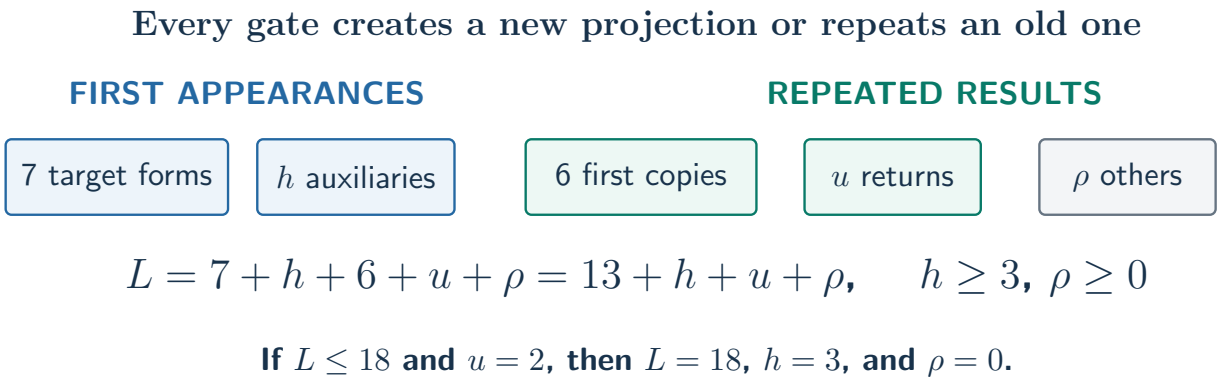


Figure 9. The exact accounting identity and its zero-slack consequence. The six first copies are unavoidable because the private outputs start with zero projection. This does not set their independent physical inputs to zero. The figure explains the event count behind Lemma 5; the original and transposed bounds are applied separately, not added.

Lemma 6. *If $u = v = 0$, then $L \geq 20$.*

Proof. Only sources can update the central row, and only sources or the central row can update private outputs. Write the central row as $t + z$. The five target-row groups counted below are disjoint, so their compulsory costs may be added. Let m_{be} count the coordinates in $\{b, e\}$ that are ever present in z, and let m_{ad} count those in $\{a, d\}$. Set $m = m_{be} + m_{ad}$. Each visited coordinate in these sets must be toggled on and off; c, f must each be toggled an odd number of times. Central updates cost at least $2 + 2m$. Outputs y_0, y_1 require at least two central transmissions in total and direct additions for the $4 - m$ unvisited required coordinates, costing at least $6 - m$. Outputs y_2, y_3 each cost at least two: without central transmissions they require two distinct sources, and with them their number of central transmissions is positive and even. Output y_4 costs at least $\min(4, 6 - m)$, by separating the absence of central transmissions from the case of at least two transmissions. Similarly y_5 costs at least $4 - m_{be}$. These target groups are disjoint, so their costs sum to $16 + m_{ad} + \min(4, 6 - m_{be} - m_{ad})$. For $m \leq 2$ this is $20 + m_{ad} \geq 20$; for $m \geq 2$ it is $22 - m_{be} \geq 20$. □

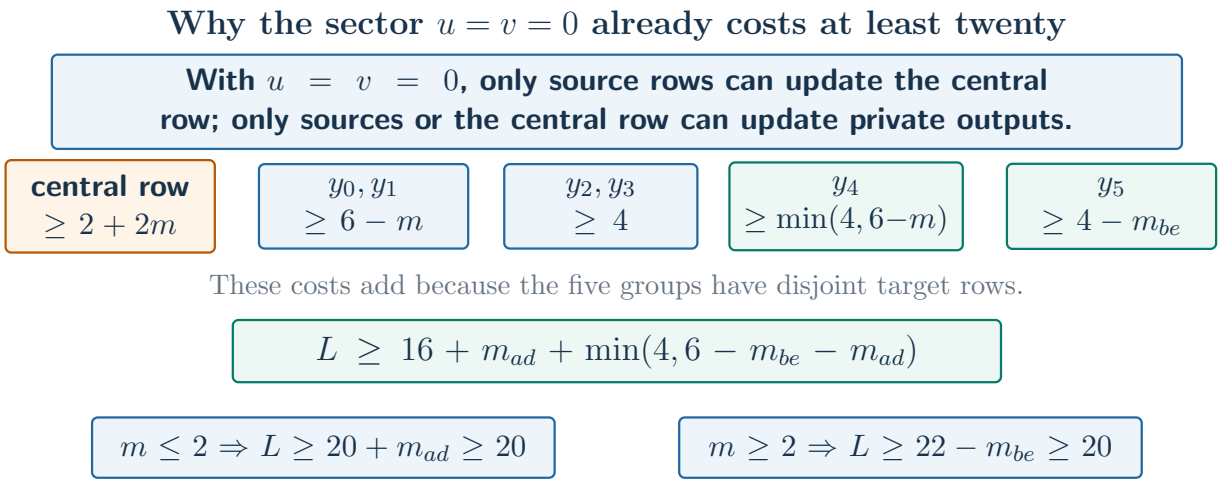


Figure 10. The additive cost count in Lemma 6. The five boxes correspond to disjoint target-row groups, so their lower bounds add. The final split into $m \leq 2$ and $m \geq 2$ shows directly that the sector $u = v = 0$ cannot contain a program of length at most eighteen.

6 Covering sectors and auxiliary catalogues

In normalized coordinates let $A \subseteq \{0, \ldots, 5\}$ be the sources that must actually change, and $R \subseteq \{0, \ldots, 5\}$ the private outputs allowed to control. Members of R need not actually control. Put

$$p = |A|, \qquad q = |R|.$$

Thus p is the number of selected changing sources and q is the number of private outputs placed in the searchable core. Every source outside A stays fixed, and every output outside R is passive. An *orientation* means either the original problem or its length-preserving relabelled transpose.

Definition 7 (Activity sector)**.** *An activity sector is the data* $(\textit{orientation}, A, R)$*. It searches words in which exactly the sources in* A *are required to change at least once and only private outputs in* R *are permitted to act as controls. The set* R *is an allowance: some of its members may never control.*

All possibilities remaining after Lemmas 5 and 6 are covered by the following families:

Actual case	Orientation	$\lvert A\rvert$	$\lvert R\rvert$	Sectors
$u = 2$, $v \leq 2$	original	2	2	225
$v = 2$, $u \leq 1$	transposed	2	1	90
$u = v = 1$	original	1	1	36
$(u, v) = (1, 0)$ or $(0, 1)$	both	1	0	12

Pad the set of actual controlling outputs to the displayed size as needed; this enlarges a sector. Sources in A must actually change, which is checked at termination. The 363 sectors need not be disjoint.

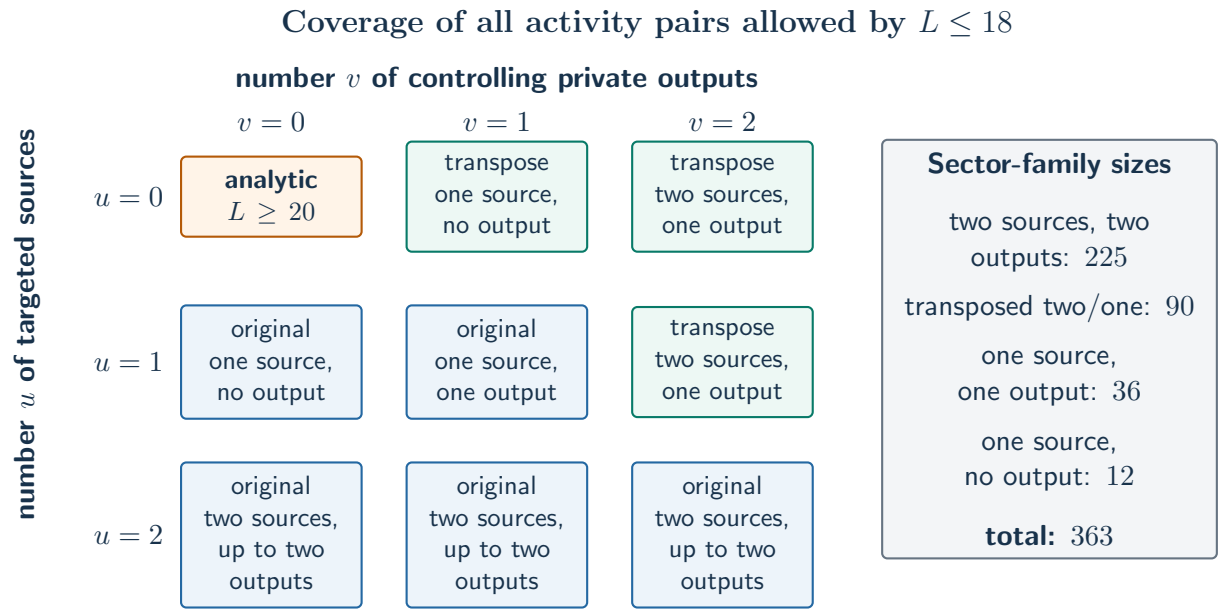


Figure 11. The activity-pair coverage. Padding the allowed set of controlling outputs enlarges a sector. Transposition is used to cover activity patterns with two such outputs; it is a length-preserving correspondence between problems. This proves only coverage by sector parameters; it does not prove completeness of the later normalized state search or the finite exclusion.

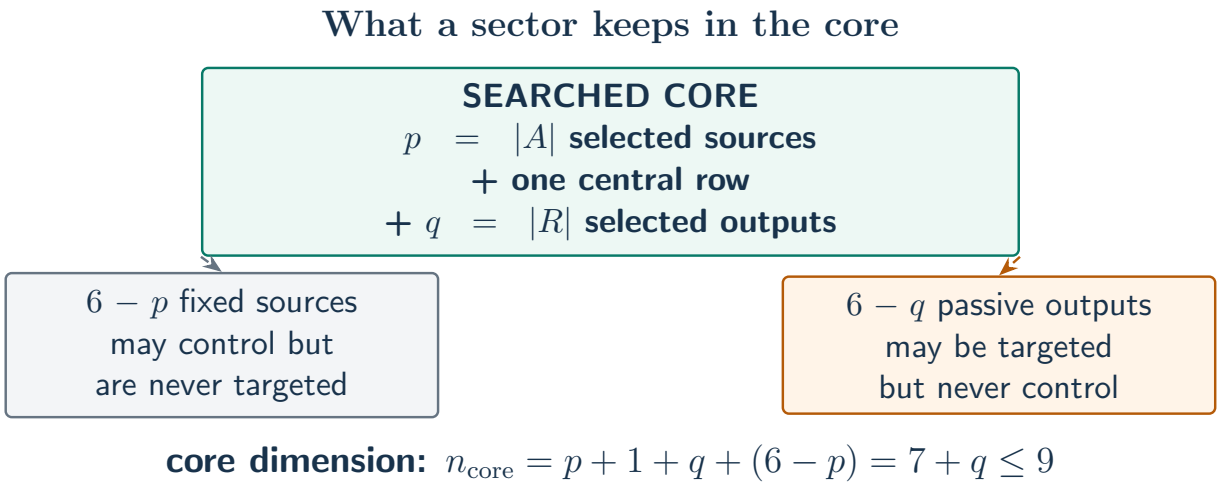


Figure 12. Anatomy of one sector. The searchable core contains the p sources in A, the central row, and the q private outputs in R. The other $6 - p$ sources are fixed controls; the other $6 - q$ outputs are passive targets and never control. The sector records permission to control, not a claim that every member of R actually does.

Sector and catalogue notation.

Symbol	Meaning
A	selected source indices that must actually be targeted
R	private-output indices permitted to act as controls
p, q	cardinalities $p = \lvert A\rvert$ and $q = \lvert R\rvert$
Ω	all possible auxiliary projected forms outside $P \cup F \cup \{0\}$
B_0	one candidate auxiliary set tested by catalogue closure
B_*	the complete auxiliary set of a hypothetical short program
$\mathcal{C}_k$	all closure-valid auxiliary sets of cardinality k

For the target list F in either orientation, set $\Omega = \mathbb{F}_2^7 \setminus (P \cup F \cup \{0\})$, with $|\Omega| = 113$. Here Ω is the universe of possible nonzero auxiliary projected forms. For $k = 3, 4$, enumerate every k-element subset $B_0 \subseteq \Omega$. Starting from P, repeatedly add an element of $F \cup B_0$ when it is the sum of two available forms. Retain B_0 exactly when this closure reaches all of $F \cup B_0$; call the resulting catalogue $\mathcal{C}_k$. To see necessity for the short programs under consideration, take their complete auxiliary set B_*. The budget gives $|B_*| = h \in \{3, 4\}$. Order $F \cup B_*$ by first appearance. At its first appearance, each form is the sum of two forms already present in $P \cup F \cup B_*$. The closure process can therefore reproduce this order and reaches every element, so $B_* \in \mathcal{C}_{|B_*|}$. Conversely, a closure sequence is itself a free XOR construction of all listed forms. Monotonicity makes the final closure independent of the order in which available additions are selected. Equivalently, if x_i denotes the ith form in first-appearance order, there are earlier available parent forms u_i and v_i with $x_i = u_i + v_i$. The implementation examines all $\binom{113}{3} = 234136$ triples and $\binom{113}{4} = 6438740$ quadruples. It obtains

Orientation	$\lvert\mathcal{C}_3\rvert$	$\lvert\mathcal{C}_4\rvert$
Original	298	21687
Transposed	298	21823

At a search prefix the observed auxiliary forms need only be contained in some feasible full set. Capacity four permits subsets of both triples and quadruples. These catalogues are regenerated from F by the verifier; their displayed sizes are diagnostics, not externally trusted inputs.

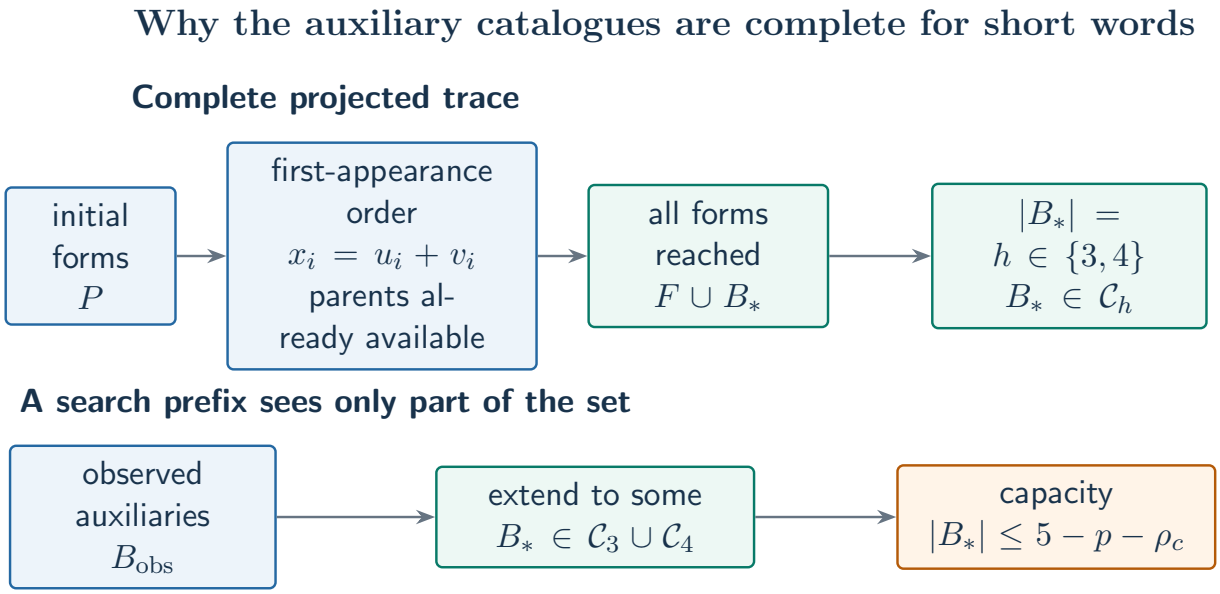


Figure 13. Why the catalogue test is necessary for a short program. The complete projected trace orders every required or auxiliary form by first appearance, proving that its full auxiliary set belongs to $\mathcal{C}_3$ or $\mathcal{C}_4$. A search prefix need only extend to such a set within its remaining capacity. The verifier still enumerates every candidate triple and quadruple; the picture is not a substitute for that enumeration.

7 The zero-slack clean-carrier argument

This section applies only to $p = |A| = 2$, where $h = 3$ and $\rho = 0$. Write a full row as (ϕ, μ), where $\phi \in \mathbb{F}_2^7$ is its projected part and $\mu \in \mathbb{F}_2^6$ records its six private-input coefficients. Call the row *clean* when $\mu = 0$, and *dirty* otherwise. Thus clean does not mean that ϕ is a basis form; it means only that no private coefficient contaminates the row.

Lemma 8. *Before a private output j first acquires nonzero projection, its row is its original private unit vector, and that private bit is absent from every other row.*

Proof. A zero-to-zero projected update would be an unbudgeted repeat, so this output has not yet been targeted. Its first use as control would export its fresh private bit while leaving the target projection unchanged. That is neither a first nonzero copy nor a full source restoration, and would again be an unbudgeted repeat. □

Lemma 9. *After the first occurrence of $\phi \notin P \cup \{0\}$, another occurrence can only be a first nonzero copy to an uninitialized private output. It adds that output's fresh private bit. Consequently a dirty first occurrence cannot later produce a clean occurrence, and a clean occurrence cannot be recreated after it disappears.*

Proof. A source restoration gives a member of P, so the only admissible repeat of ϕ is a first nonzero copy. Let $e_j \in \mathbb{F}_2^6$ be the private unit vector belonging to output j. By Lemma 8, such a copy sends (ϕ, μ) to $(\phi, \mu + e_j)$ with the fresh bit e_j absent from μ. Successive copies of this same projection cannot cancel those fresh bits. To reach the final private row (ϕ, e_j), either ϕ is first created at its final update on j, or a clean ϕ is copied into uninitialized j. In particular, a first occurrence carrying e_j on a different row implies that j was already initialized, forbidding the latter copy. □

The central target f_6 must therefore first occur on its own row in its final full form and must remain there. A controlling private output whose required projection has already occurred must either be final, or still have zero projection with a clean carrier currently present on a core row. For a passive output whose required projection has occurred in the core history, that history must contain its clean form. These are necessary conditions only. Fixed sources cannot carry a clean nonunit form. A passive output cannot carry a clean form at all: its own private bit cannot enter any controlling row and hence cannot be cancelled there. If a target projection were first created on a passive output, it could not subsequently return to the core without an unbudgeted repeat. Removing passive updates therefore does not invalidate the stated necessary conditions.

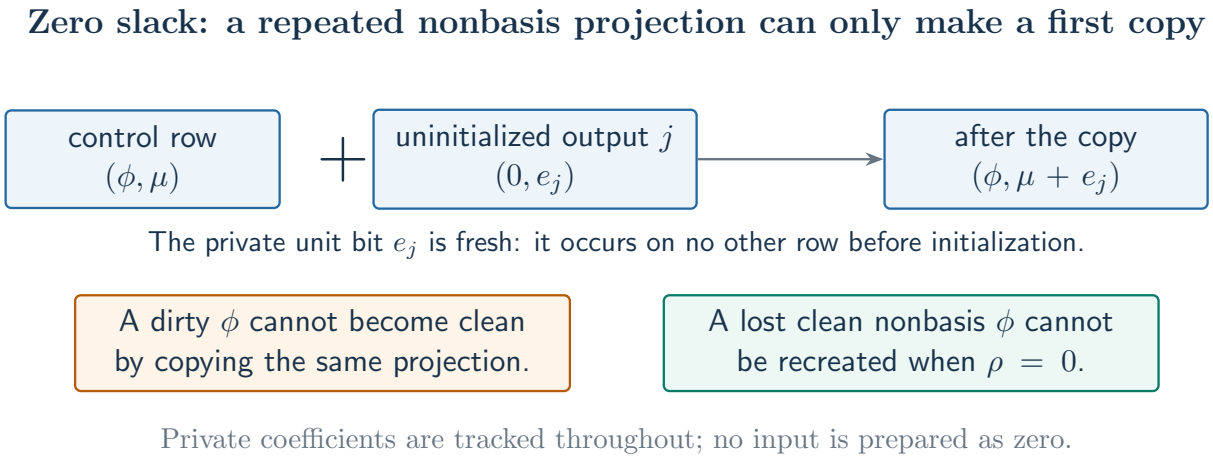


Figure 14. The fresh-private-bit mechanism behind Lemmas 8 and 9. In the zero-slack case, the only allowed repeat of a nonbasis projection is a first copy into an uninitialized output, which adds a private bit that cannot cancel. These are necessary pruning conditions only, not a sufficient characterization of valid histories.

8 Exact elimination of passive targets

Fix $p = |A|$ and $q = |R|$. A core history consists of updates to the p changed sources, the central row, and the q potentially controlling outputs. Denote this gate sequence by γ and its length by $K = |\gamma|$. Its rows retain $n_{\text{core}} = 7 + q \leq 9$ bits, including the private bits of those q outputs. Together with the $6 - p$ fixed sources, these rows form an invertible $n_{\text{core}} \times n_{\text{core}}$ system at every step. Private bits of passive outputs cannot enter this system because those outputs never control; their omission is justified by the sector alphabet.

Deleting passive-target gates is only a compression of the history; their cost is charged back exactly. Let $D(\gamma)$ be the set of every distinct retained row value that occurs on an allowable control during γ, including the initial basis. Embed each seven-bit f_j into $\mathbb{F}_2^{n_{\text{core}}}$ by appending zero private coefficients. Define

$$\delta_D(f) = \min\{|B| : B \subseteq D, \bigoplus_{b \in B} b = f\}, \qquad E(D) = \sum_{j \notin R} \delta_D(f_j).$$

Here $\delta_D(f)$ is the smallest number of historical controls whose XOR is f, and $E(D)$ is the total minimum completion cost of all $6 - q$ passive outputs.

Symbol	Meaning
γ	the ordered sequence of core-target gates after passive gates are erased
K	core length $\lvert\gamma\rvert$
n_{core}	retained row width $7+q$
$D(\gamma)$	all distinct retained control values available during γ
$\delta_D(f)$	minimum subset size in D whose XOR is f
$E(D)$	sum of these minimum costs over all passive outputs

The initial basis ensures existence; repeated copies of one form cancel and can be removed.

Lemma 10. *For a core history γ ending at all prescribed core target rows, the minimum cost of a full circuit extending that history is $\lvert\gamma\rvert + E(D(\gamma))$.*

Proof. Every update of a passive row adds a member of D, and its total added value must be exactly f_j, including cancellation of any private contamination. This proves the lower bound. Conversely, choose a minimum subset for each j and insert each transmission at a time when its control value is available. Passive updates do not affect the core or one another, so all insertions coexist and give the required full operator. □

The endpoint hypothesis in this lemma is essential.

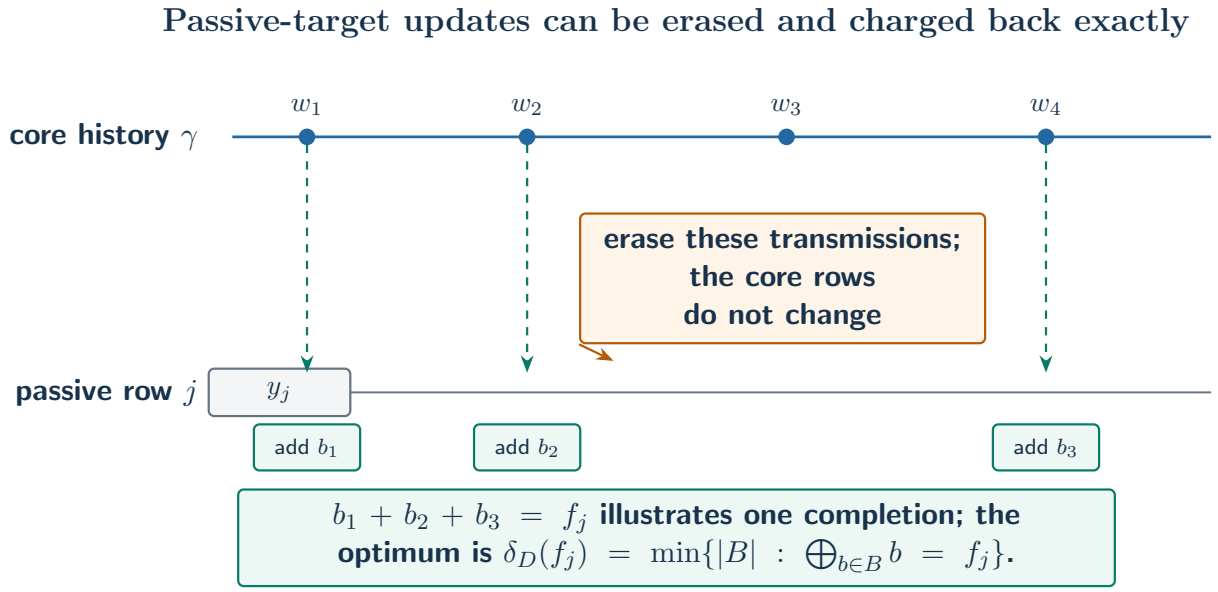


Figure 15. A schematic passive-output history. Dashed transmissions may be erased without changing the full core rows, and inserted at the moments their control values exist. The displayed three-term XOR is illustrative; the minimization uses every subset of the actual historical control set D. The exact conclusion is conditional on a fixed core history with the prescribed endpoint; the picture does not show that all such core histories have been enumerated.

The verifier computes δ_D by exact zero-one subset dynamic programming. For a processed prefix of D, let $c(x)$ be the smallest subset size with XOR x. Start with $c(0) = 0$ and $c(x) = \infty$ otherwise; for each $w \in D$ update

$$c_{\text{new}}(x) = \min(c_{\text{old}}(x), 1 + c_{\text{old}}(x + w)).$$

Reading both old values in each pair $\{x, x+w\}$ prevents reuse of w. Induction over D proves the recurrence. There are at most $2^9 = 512$ values. Each of the $6-q$ passive outputs needs at least one operation, so a candidate of cost at most eighteen has core length

$$K \le 12 + q. \tag{6}$$

9 A remaining-cost bound

At a search state, let X be the current retained matrix and Z its target. For each row index i with $X_i \neq Z_i$, the vector $X_i + Z_i$ is the increment that must eventually be added to that row. Let r be the number of differing rows, and let b be the number of distinct required increments that occur as rows in neither endpoint matrix. These definitions are local to the following lemma.

Lemma 11. *For invertible binary matrices X, Z, let r count differing rows and let b count distinct increments $X_i + Z_i$ for those rows that occur as rows in neither X nor Z. Every CNOT path between them has length at least $r + \lceil b/2 \rceil$.*

Proof. Write its length as $r+k$. At most k required rows can be updated more than once. Removing their increments loses at most k distinct values. Each remaining increment must be available on a control at the unique update of its target. Being absent from both endpoints, it must have been created as an intermediate value. Reserve the last update of each of the r differing rows; these create only final rows. All nonendpoint intermediate values are created among the other k operations. Hence $b - k \le k$. □

Include fixed sources among the rows at both endpoints when applying this to a core. Let z_A be the number of chosen sources that have never changed. Each such source still equals its target but must change and return. Thus another bound is $r + 2z_A$. We use the maximum of the two valid bounds, not their sum:

$$B_{\mathrm{rem}} = \max\{r + 2z_A, r + \lceil b/2 \rceil\}.$$

The symbol B_{rem} therefore denotes a certified lower bound on the number of future core gates still required.

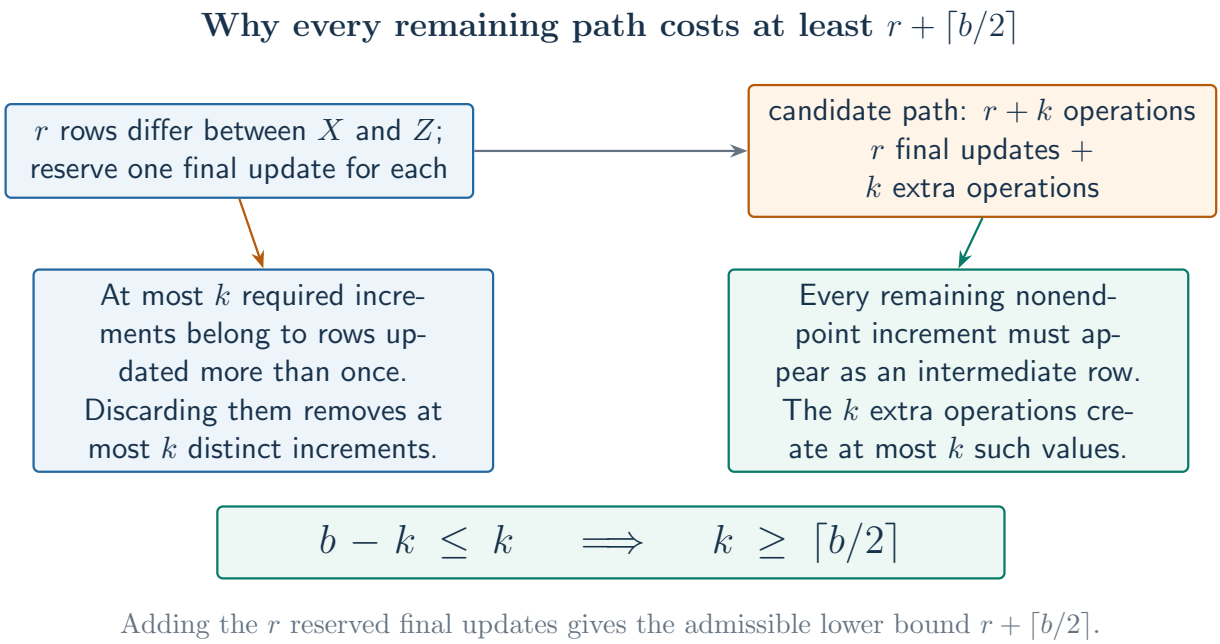


Figure 16. The counting argument for Lemma 11. Here b counts distinct required increments absent from both endpoint row sets. The k extra operations bound both the discarded increments and the newly created intermediate values. This is an admissible lower bound on the remaining cost, not a claim that the bound equals the exact CNOT distance.

10 Finite search and completeness

The verifier does not enumerate all 13×13 matrices or all length-18 gate words. For a fixed sector it enumerates only core histories, compresses each prefix to a label containing all information relevant to the future, and finally adds the exact passive completion cost.

For each sector, order core rows as chosen sources, central row, and chosen controlling outputs. Choose fixed orderings $A = (s_0, \ldots, s_{p-1})$ and $R = (j_0, \ldots, j_{q-1})$. Let e_i denote the ith standard basis vector of $\mathbb{F}_2^{n_{\text{core}}}$. The initial core rows are e_{s_i}, e_6, and $e_{7+\ell}$; their target rows are e_{s_i}, f_6, and $e_{7+\ell} + f_{j_\ell}$, where $0 \le i < p$ and $0 \le \ell < q$. Each f_j is padded with q trailing zeros. Fixed controls are e_s for $s \notin A$. The retained coordinates are the seven projected coordinates followed by the q private bits belonging to R. Thus a full retained row is a $n_{\text{core}} = 7 + q$ bit mask. At every time, the core rows together with the fixed source rows are the image of the initial n_{core} basis rows under invertible row operations; in particular they remain a basis and no genuine core update can create the zero full row.

At each fixed core length store the complete label

$$\lambda = (\mathcal{R}, D, C, T, I, \rho_c),$$

where $\mathcal{R}$ is the tuple of current full core rows, D the historical full control set including the initial basis, C the mask of changed sources, T the mask of credited full source restorations, I the mask of first nonzero private-output initializations, and ρ_c the count of other projected repeats observed in the core. The masks C and T have one bit for each of the p selected sources; I has one bit for each of the q selected private outputs. Here D contains every distinct full row value that has appeared in the core history. Each such value was available as a control at the time it appeared, although only current rows, never merely historical members of D, may control a future core gate. The mathematical fields correspond directly to the implementation:

Field	C++ member	Information retained
$\mathcal{R}$	`row`	current ordered core rows
D	`appeared`	all distinct full row values seen so far
C	`changed`	chosen sources targeted at least once
T	`returned`	one credited full restoration per chosen source
I	`copied`	one credited first nonzero initialization per core output
ρ_c	`extra`	all other repeated projected results in the core

The sector, its orientation, A, R, and the core length are fixed outside the label.

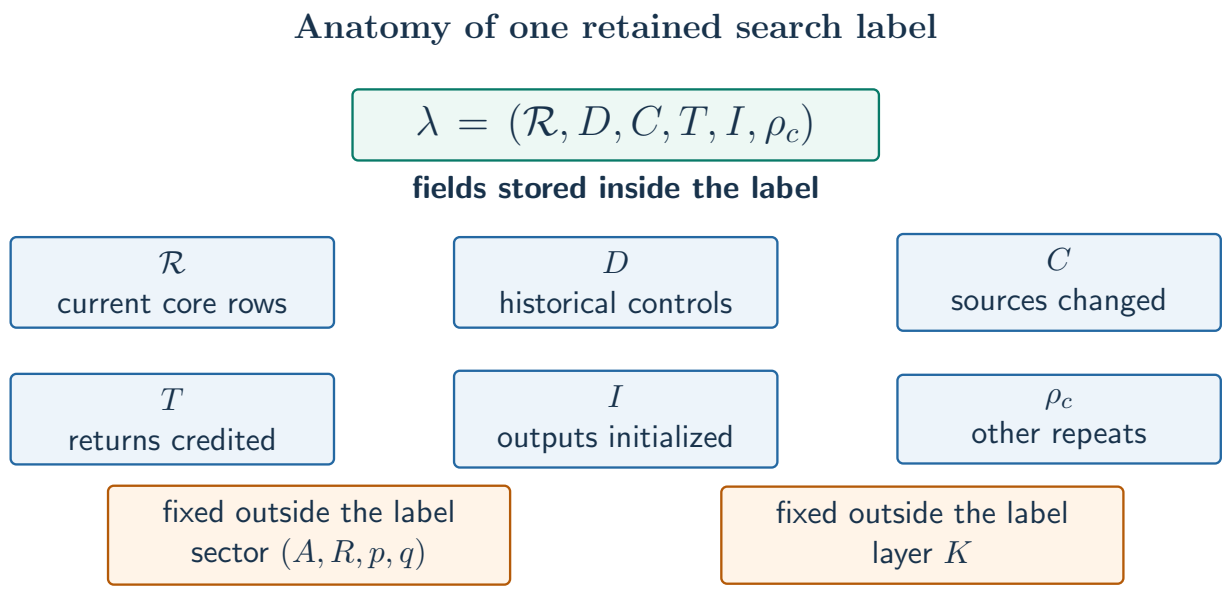


Figure 17. What one search label remembers. The current rows $\mathcal{R}$ determine legal next gates; the historical set D determines passive completion; C, T, I, and ρ_c retain the finite budget information. Sector data (A, R, p, q) and the layer number K are fixed outside the label. Equal labels at the same layer have identical admissible futures.

Enumerate every distinct control-target pair with target in the core and control in the core or fixed sources. Reject a zero full new row. Credit the first full restoration of a chosen source, and the first nonzero projected initialization of a chosen private output. The choice of first rather than last restoration is harmless: if one source returns to its basis row r_s times, exactly $r_s - 1$ of those returns remain additional repeats whichever return receives the credit. If a projected result is in $P \cup \{0\} \cup \pi(D)$ and neither event is credited, increment ρ_c and require $\rho_c \le 2 - p$. Here $\pi(D) = \{\pi(d) : d \in D\}$ is the set of projections of historical control values.

Let B_{obs} be the set of auxiliary projected forms observed in the current core prefix. Require B_{obs} to extend to a feasible catalogue set of size at most $5 - p - \rho_c$. Update the rows and D; when $p = 2$, apply the clean-carrier conditions. Finally require the current length plus B_{rem} not to exceed (6). Historical values in D are never used as current core controls.

The direction of every rejection test is important. For a putative word of length at most eighteen, the exact budget with $|A| = p$ gives

$$h + \rho \le 5 - p, \qquad \rho \le 2 - p, \tag{7}$$

because $h \geq 3$. Erasing passive updates can hide earlier projected occurrences, so the core count ρ_c is at most the global extra-repeat count ρ. The auxiliary forms seen in the core are likewise a subset of the global auxiliary set B_*. By the first-appearance argument above, B_* belongs to a feasible catalogue, and $|B_*| = h \leq 5 - p - \rho \leq 5 - p - \rho_c$. Hence both the repeat limit and the catalogue extension test are relaxations of (7), not additional restrictions on a true candidate. When $p = 2$, Lemmas 8 and 9 make the clean-carrier tests necessary. Finally, Lemma 11 and the untouched-source bound are lower bounds on the actual remaining core suffix, so pruning when the current length plus B_{rem} exceeds (6) is also safe.

At every layer, inspect labels with all core rows final and every chosen source marked changed. Such a label is a candidate exactly when $K + E(D) \leq 18$. Equal labels at an equal layer can be merged because all future transitions and tests depend only on the current rows, D, the four counters or masks, and the fixed sector data. The terminal cost depends only on D, the final rows, and the changed-source mask. Thus two equal labels at the same layer have identical admissible continuations and terminal cost. Hash collisions are resolved by complete label equality, so hashing does not identify unequal labels.

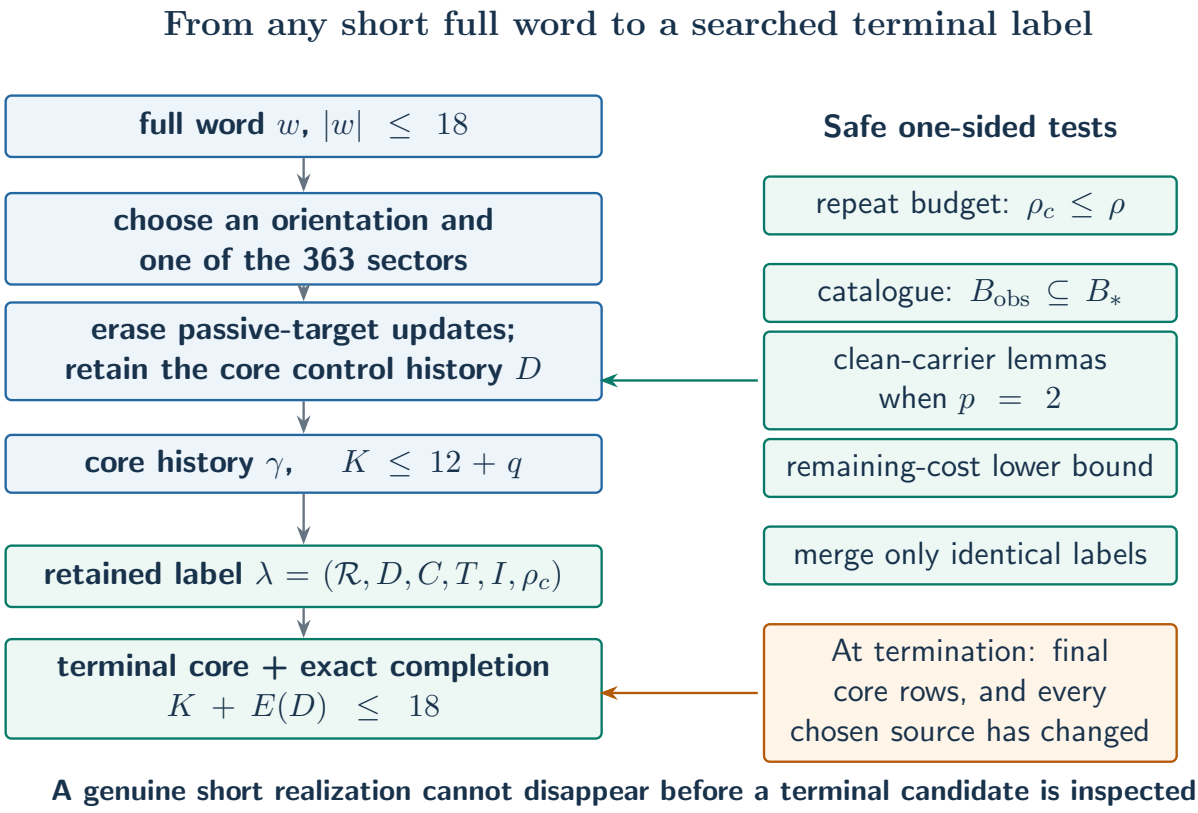


Figure 18. The completeness pipeline formalized in Lemma 12. A hypothetical short full word is assigned to a sector, stripped to its core history, and followed through the retained-label search. Every rejection test is one-sided, and passive gates are charged back exactly by $E(D)$. Thus a true short realization would produce a terminal candidate inspected by the verifier.

Lemma 12. *Any realization of M with at most eighteen gates yields a terminal candidate in at least one searched sector.*

Proof. Let w be such a realization.

Sector and orientation. The two budget bounds give $u, v \leq 2$, while Lemma 6 excludes $u = v = 0$. The activity-pair table therefore selects one of the 363 sectors, possibly after the length-

preserving transpose correspondence. Its set A is exactly the sources targeted by the oriented word. Its set R contains every private output used as a control; padding R only moves additional outputs into the core and cannot invalidate the word.

Erasure and representation. Delete every gate whose target is outside the core. Such a passive coordinate never controls, so these deletions leave the entire sequence of core rows unchanged. Sources outside A are never targeted and remain fixed controls. Consequently every surviving gate has a core target and a control that is either a current core row or a fixed source, exactly the alphabet enumerated by the search. Passive private bits never enter a core row, so restricting the surviving rows to the $n_{\mathrm{core}} = 7+q$ retained coordinates loses no information about this core history. If its length is K, the deleted part contains at least one update for each of the $6-q$ passive outputs; hence $K \le 18-(6-q) = 12+q$, as in (6).

Induction over the core history. The initial search label is precisely the initial retained basis, with that basis recorded in D and all masks and counters zero. Suppose the label of a genuine prefix survives at its layer. The next genuine gate is among the enumerated control-target pairs. Its new full row is nonzero by invertibility. Updating $\mathcal{R}$ and adjoining the new value to D reproduces the next genuine prefix. The definitions of C, T, I, and ρ_c reproduce the credited change, restoration, initialization, and repeat events; crediting the first rather than the final restoration only changes which one of several returns is exempted, not their total number. The repeat and catalogue tests are safe by (7) and the catalogue first-appearance argument. In the zero-slack case the clean-carrier tests are necessary by Lemmas 8 and 9. The remaining-cost test is no larger than the genuine suffix. Thus the successor is retained, unless it is merged with an equal label; label sufficiency then preserves the same continuation. Induction reaches the complete erased core history.

Terminal cost. The final core rows are their prescribed full targets, and every member of A has been targeted, so the resulting label passes the terminal test. For each passive output, the controls used by the deleted gates occur in D and their XOR is its required increment. Cancelling duplicate controls gives a subset of D no larger than the deleted list. Therefore $E(D)$ is at most the number of deleted gates; by Lemma 10, it is also the exact minimum passive completion cost. Hence $K + E(D) \le |w| \le 18$, and the label is a terminal candidate. □

Finite exclusion. A fresh full run of the supplied implementation completes all sectors:

Family	Sectors	Layer labels	Terminal labels
Two sources, two outputs	225	240830308	21684
Transposed: two sources, one output	90	3175146	1848
One source, one output	36	62738391	65679
One source, no controlling outputs	12	179149	1305
Total	363	306922994	90516

No terminal label has total cost at most eighteen. These are sums of layer sizes, not counts of distinct matrices or circuits. A sentinel cost of 999 means no terminal was found, not a length-999 witness. The successful exit is zero with the final marker `VERIFIED_FIXED_MATRIX_MINIMUM_19`. The implementation imposes no heuristic truncation of the searched layers. An interrupted or failed run does not establish the exclusion. Together with Lemma 12, this exhaustive result supplies the lower bound. Its evidentiary form is source code plus a reported rerun, not an independently checkable proof certificate.

Completion of Theorem 2. A circuit of cost at most eighteen would contradict finite exclusion by Lemma 12. The explicit nineteen-gate circuit supplies the opposite inequality. □

11 Reproducibility and formalization boundary

The proof separates a general mathematical reduction, its concrete C++ implementation, and the finite execution. Lean checks the general reduction and derives the exact minimum from the explicit finite premise below. The concrete tables and exclusion are supported by the external audits and run evidence supplied under `anc/`; they are not replayed by Lean.

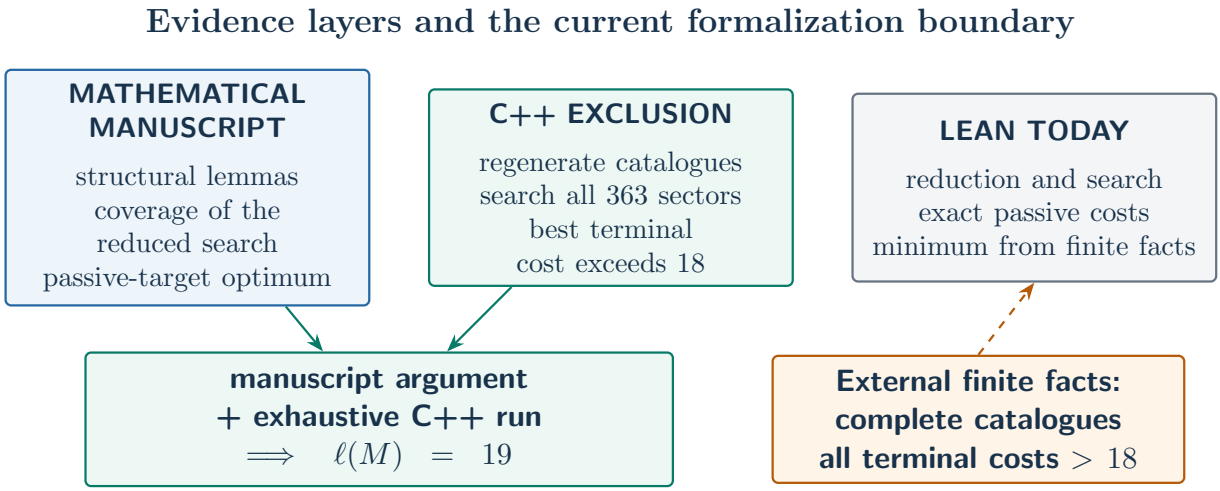


Figure 19. The updated verification boundary. Lean checks the general reduction and the conditional minimum theorem. Concrete catalogue completeness and finite exclusion are supported externally; the C++ implementation is audited rather than formally verified. No unconditional kernel proof is claimed.

Finite C++ exclusion and implementation audit

The source file `hgp13_verify.cpp` regenerates all catalogues and sectors and requires no data files or third-party C++ libraries. Compile and run with

```
g++ -O3 -std=c++17 -pthread hgp13_verify.cpp -o hgp13_verify
./hgp13_verify 3
```

The rerun used GCC 11.4.0 in a Linux container with networking disabled. Its source SHA-256 is

`2174f445df534455c30e9ae7b54b8dfff4fdbdce3ff4eaf6b7d7e557fa3f8e4c.`

The accompanying evidence contains the full run log and source identity. An additional full run compiled that exact source with the same GCC flags in the pinned network-disabled container. The wrapper recorded the source and binary SHA-256 values, command, exit code zero, and new transcript hash. All 363 case records and family totals match the preserved run; the manifest and audit are in `anc/evidence/full_provenance_2026-09-27/`. This repeats the same implementation rather than independently proving its semantics. The reproducible script `anc/scripts/audit_full_run.py` checks all 363 case records, layer and family sums, two comparison transcripts, and equality of the saved exported catalogue key sets with the literal Lean lookup trees. An independent exhaustive enumerator, which does not include the verifier source and adds all currently derivable forms in simultaneous rounds, reproduces the same four key sets and catalogue counts. This is an external cross-check of the concrete tables, not a kernel proof of their completeness or of the C++ generator semantics. Separate executable exports from C++ and Lean agree byte for byte on the orientation, source and relay tuple of each of the 363 sectors. Running the abstract Lean transition

once for every sector yields 363 first-layer sizes that all agree with the saved C++ layer counts. A source-hash-pinned instrumented C++ diagnostic and a separate Lean export also agree on all 8,610 complete first-layer state records, including rows, history bits, and counters. For sectors 234 and 359, source-hash-pinned diagnostics exported every layer: respectively 6,668 and 2,208 complete state records agree as sets with the candidate graphs whose transitions are checked in Lean. These comparisons cover two sectors and do not establish later-layer correspondence for the other 361 sectors. The supplementary source audit records bounds for the packed arrays and catalogue keys, the layer and 0/1-cost loop invariants, and the worker join conditions. A separate UBSan diagnostic on sector 357 matched its preserved full-run record; it does not certify the other sectors. The pinned terminal-cost routine also agrees with a separate shortest-path BFS oracle on 1,518 deterministic 7–9-bit histories and all 453,376 resulting cost entries; this bounded comparison is not a formal proof of the C++ loop. This agreement is repeatability evidence for the same implementation. The log itself does not cryptographically attest which binary produced it, and the search result is not a Lean kernel certificate.

Lean theorem and its finite premise

The Lean development pins version 4.34.0 and uses its bundled standard library. It specifies bit-vector rows, legal gates, full matrix execution, the nineteen-gate witness, and the exact-minimum proposition. It also develops the preservation of full core rows under erasure of passive-target gates and exact row-update accounting. The kernel-checked development now also proves free XOR complexity ten in both orientations, the necessity of three distinct auxiliary projected values, full-word transposition, and the unconditional bounds $L \geq 16 + u$ and $L \geq 16 + v$. The exact budget $L = 13 + h + u + \rho$ and its short-word consequences are checked in both orientations. For the concrete transpose free-XOR lemma, an inductive first-appearance argument reduces the certificate to 198 ordered pairs of auxiliary values; the kernel checks both their closed-set witnesses and coverage of all seven-bit pairs. This small certificate does not certify the main CNOT exclusion. The general rectangular free-XOR transposition principle is not yet formalized; its concrete consequence here is proved independently. The kernel also proves $L \geq 20$ when $u = v = 0$, by counting actual gates and tracking coefficient parities. It verifies that the 363 sector parameter records are distinct and cover every putative legal word of length at most eighteen, preserving the chosen orientation's full operator and length. The abstract auxiliary-catalogue lemma is checked: saturation is equivalent to a valid first-appearance program, and every full realization supplies an accepted auxiliary set in either orientation. The size bounds and abstract subset test for partial histories are also checked. The concrete table enumeration and its reported cardinalities remain outside this certificate. At the saturated two-source budget, the kernel proves uniqueness of the mandatory repeats and the fresh-private-bit invariant at every prefix. It also proves that a clean nonbasis value must be globally new, and that a repeated final nonbasis row is copied from its exact clean goal into an intact private output. All three semantic core-history conditions are now proved in both orientations: the seen central goal is already final; an unfinished kept output has an intact row and a current exact clean carrier; and a seen passive goal has its exact clean form in core history. Erasure of passive-target gates preserves and reflects this full-row history. The exact passive-cost theorem is also proved: the 0/1 recurrence computes the minimum over distinct historical forms, all passive minima can be attained simultaneously by legal insertions, and no completion with the same core is shorter. Existence of the required representations follows from the seven retained initial basis rows in both orientations. For rows supported on a sector's retained coordinates, `PackedCostRefinement.cost_pack_eq` proves that this exact recurrence has the same value on full thirteen-bit rows and their packed integer masks. The correspondence of the imperative C++ array update to that recurrence is reviewed

separately, not verified by Lean. The theorem `SectorPackedCost.actual_core_cost_pack_eq` instantiates this equality for the complete retained history of every compatible full program and each of its passive target differences. It also checks that those differences have no discarded-coordinate bits. The stronger `SectorPackedCost.indexed_history_cost_pack_eq` applies to every indexed sector path, including arbitrary terminal paths quantified over in the external finite premise. Its complete full-row history is supported, and its packed 0/1 cost equals the full-row cost. `CostSetInvariant.cost_eq_of_mem_iff` proves that exact cost depends only on which historical forms occurred. Its packed corollary shows that deleting repeated supported forms leaves the cost unchanged, matching the mathematical role of the C++ history bitset. For every indexed sector path, the theorem `IndexedHistoryBits.indexed_history_bit_iff` identifies each bit in the abstract search history exactly with membership of the corresponding supported full row in the complete retained history. Define $\mathcal{H}_s(h)$ as the supported full rows whose packed indices are set in history bitset h. The theorem `StateHistoryCost.indexed_cost_eq_decoded` proves, for every indexed path and every passive target, that its full historical 0/1 cost equals the cost computed from $\mathcal{H}_s(h)$ at the terminal state. Thus the mathematical terminal cost depends only on the complete state. The imperative C++ loop remains an external implementation of that packed recurrence. The two remaining-cost bounds and their maximum are proved for full 13-coordinate states, with sound pruning and an application to actual core suffixes. Lean now also proves that any legal short full word maps to an accepted path of the abstract search when the concrete prefix lookup is complete. For every sector, the exact passive cost of that path's full-row history is bounded by the length of the original word. The theorem `ConcreteFiniteBoundary.exact_minimum_of_external_finite_result` therefore concludes the minimum nineteen from one explicitly named premise, `ExternalFiniteResult`. Here P_s denotes the passive-output indices of sector s, and M_s its oriented target matrix. The premise asserts complete concrete prefix lookups and, for every sector s and depth k within its cap, that each terminal state z in the deduplicated abstract frontier $F_{s,k}$ satisfies

$$k + \sum_{j \in P_s} \mathrm{cost}_{\mathcal{H}_s(z.\mathrm{history})}(e_j \oplus M_s[j]) > 18.$$

`FiniteFrontierBoundary.no_cheap_terminal_of_frontier` transfers this state-and-depth condition to the path-quantified premise of the general lower-bound theorem. The unconditional proposition is

```
def NoShortProgram : Prop :=
  forall p : Program, Legal p -> p.length <= 18 ->
    Not (run identity p = target)
```

At this draft stage it is an open Lean proposition, not an added axiom. The conditional theorem does not make the external finite premise kernel-verified. Its data and implementation correspondence are reviewed in `anc/evidence/cpp_refinement_audit.md`; a static review is not a formal proof of C++ semantics. The build and transitive-axiom audit record which mathematical statements Lean accepts. Full kernel replay of all sectors is outside this project's scope. No `sorry` or native reduction trust axiom occurs in the accepted development.

Acknowledgements

The author acknowledges OpenAI, the Astra AI assistant, and the CayleyPy project. AI assistance was used in preparing the manuscript, examining the computational evidence, and developing the Lean formalization with its explicit external finite premise. Responsibility for the final mathematical claims and their presentation rests with the author.

References

[1] K. N. Patel, I. L. Markov, and J. P. Hayes, "Optimal synthesis of linear reversible circuits," *Quantum Information and Computation* 8 (2008), 282–294. https://www.rintonpress.com/xxqic8/qic-8-34/0282-0294.pdf.

[2] A. Bu, E. Fan, and R. Joo, "Minimum Synthesis Cost of CNOT Circuits," arXiv:2408.07898 (2024). https://arxiv.org/abs/2408.07898.

[3] A. Sodhani and K. K. Parhi, "Noise-Aware Synthesis of Quantum LDPC Encoder Circuits via Two-Sided Hamming Descent," arXiv:2607.04462 (2026). https://arxiv.org/abs/2607.04462.

A Complete source bundle of the current Lean development

The complete Lean sources are supplied under `anc/lean/`, with the pinned toolchain and Lake configuration. The package also includes `anc/verify_lean.py`, source-bound build and axiom-audit evidence, the C++ run records, audit scripts, and `anc/lean-project.zip`. No compiled cache is needed. The ZIP is a separate source-package file; compiling this Prism project does not embed it as a PDF attachment. The table lists the principal modules; the source archive includes their dependencies and the full development.

Module	Role
`HGP13.lean`	Full matrix semantics, exact witness, and the target minimum-length proposition.
`Passive.lean`	Preservation of full core rows and exact row-update accounting.
`Certificate.lean`	A sound interface for complete finite layer certificates.
`FreeXor.lean`	Exact free XOR complexity ten and three necessary auxiliary values for the original masks.
`Projection.lean`	Transfer from arbitrary full-matrix programs to the seven-bit free XOR model.
`CountHelpers.lean`	List counting and duplicate-removal lemmas.
`Budget.lean`	First-appearance and repeat accounting; the bound $L \geq 16 + u$.
`ExactBudget.lean`	Exact operation budget and short-word saturation.
`Transposition.lean`	Coefficientwise transposition and relabeling of arbitrary full CNOT words.
`TransposeFree.lean`	First-appearance restriction and soundness of closed-set certificates.
`TransposeCertificate.lean`	Kernel-checked finite certificates and complete coverage of auxiliary pairs.
`TransposeTarget.lean`	Concrete transposed target and the parameter correspondence.
`TransposeBudget.lean`	Concrete transpose XOR minimum, the bound $L \geq 16 + v$, and exact budgets.
`ZeroSector.lean`	Coefficient parities and the analytic lower bound twenty for $u = v = 0$.
`SectorCover.lean`	Distinctness and complete coverage of the 363 sector parameter records, with full-word semantics.
`Catalogues.lean`	Abstract saturation completeness, actual auxiliary sets, size budgets, and the partial-history subset test.
`TightRepeats.lean`	Saturated repeat classification, fresh private bits, and local restrictions on clean copies.
`CleanPersistence.lean`	Current clean carriers, all three semantic core-history conditions, and exact history preservation under erasure.
`PassiveCost.lean`	Exact 0/1 XOR costs, pair cancellation, and total passive-cost lower bounds.

Module	Role
`PackedCostRefinement.lean`	Equality of full-row and packed-mask passive-cost recurrences for supported rows.
`SectorPackedCost.lean`	Supported retained histories and passive target differences, with exact packed costs for arbitrary indexed paths.
`CostSetInvariant.lean`	Exact cost depends only on historical set membership; duplicate packed forms can be removed.
`IndexedHistoryBits.lean`	Exact correspondence between abstract history bits and full historical rows for every indexed path.
`DepthFrontier.lean`	Exact depth-dependent breadth-first reachability after complete-state deduplication and inclusion of every accepted path.
`StateHistoryCost.lean`	Exact reconstruction of full historical passive cost from the abstract terminal state's history bits.
`FiniteFrontierBoundary.lean`	State-and-layer exclusion implies the path-quantified finite premise of the lower-bound theorem.
`PassiveInsertion.lean`	Legal historical insertions and the attained minimum completion cost for each fixed core.
`RemainingBound.lean`	Both remaining-cost bounds, their sound maximum, and application to actual core suffixes.
`FullWordPath.lean`	A short full word follows an accepted abstract search path, conditional on concrete prefix completeness.
`AbstractFiniteExclusion.lean`	General terminal-cost transfer and the minimum-length implication from explicit finite facts.
`ConcreteFiniteBoundary.lean`	The conditional theorem for the two literal original and transposed lookup tables.
`Audit.lean`	Transitive axiom checks for the accepted declarations.

"Complete source bundle" describes file coverage. The central lower-bound proposition remains open without the external finite premise, as stated in Section 11; attaching all current sources does not turn the external computation into a kernel proof.

B Full finite verifier

The following is the exact rerun source, not pseudocode.

```
// Self-contained finite certificate for ONE fixed 13 by 13 binary matrix.
// Build: g++ -O3 -std=c++17 -pthread hgp13_verify.cpp -o hgp13_verify
// Full verification: ./hgp13_verify 3
// One diagnostic case only: ./hgp13_verify --one 0
// No input files, saved catalogues, libraries beyond C++17, or network access.
#include <algorithm>
#include <array>
#include <atomic>
#include <cstdint>
#include <exception>
#include <functional>
#include <iostream>
#include <mutex>
#include <set>
#include <stdexcept>
#include <string>
#include <thread>
#include <unordered_set>
#include <vector>

using U64 = std::uint64_t;
using Mask = std::array<U64,2>;
const std::array<int,13> TARGET =
    {1,2,4,8,16,32,329,402,292,515,1030,2075,4150};
const std::array<std::array<int,7>,2> F = {{
    {73,82,3,6,27,54,100},
```

```
    {21,62,104,17,50,96,67}
}};
void require(bool condition, const char* message) {
    if (!condition) throw std::runtime_error(message);
}
bool member(const Mask& m, int x) {
    return (m[x/64] >> (x%64)) & U64(1);
}
void insert(Mask& m, int x) {
    m[x/64] |= U64(1) << (x%64);
}
std::uint32_t encode(std::vector<int> values) {
    std::sort(values.begin(),values.end());
    std::uint32_t result=0;
    for (int x: values) result=result*128+unsigned(x);
    return result;
}

// All feasible sets of exactly three or four auxiliary projected values.
// prefix[k] contains ALL subsets of all feasible sets of size <= k.
struct Catalogues {
    std::array<std::unordered_set<std::uint32_t>,2> prefix;
    std::array<U64,2> counts{};
};
bool xor_closure(const std::array<int,7>& goals,
                 const std::vector<int>& extra) {
    std::array<bool,128> present{};
    std::vector<int> available, todo(goals.begin(),goals.end());
    for (int i=0;i<7;i++) {
        present[1<<i]=true;
        available.push_back(1<<i);
    }
    todo.insert(todo.end(),extra.begin(),extra.end());
    std::size_t added=0;
    while (added<todo.size()) {
        bool progress=false;
        for (int x: todo) {
            if (present[x]) continue;
            bool ready=false;
            for (int a: available) {
                if (present[x^a]) { ready=true; break; }
            }
            if (ready) {
                present[x]=true;
                available.push_back(x);
                added++;
                progress=true;
            }
        }
        if (!progress) return false;
    }
    return true;
}
Catalogues generate_catalogues(int orientation) {
    const auto& goals=F[orientation];
    std::array<bool,128> excluded{};
    excluded[0]=true;
    for (int i=0;i<7;i++) excluded[1<<i]=true;
    for (int x: goals) excluded[x]=true;
    std::vector<int> options;
    for (int x=1;x<128;x++) if (!excluded[x]) options.push_back(x);
    require(options.size()==113,"Wrong auxiliary domain");
    Catalogues out;
    for (int k=3;k<=4;k++) {
        std::vector<int> chosen;
        std::function<void(int)> visit = [&](int first) {
            if (int(chosen.size())==k) {
                if (!xor_closure(goals,chosen)) return;
                out.counts[k-3]++;
                for (unsigned bits=0;bits<(1u<<k);bits++) {
                    std::vector<int> subset;
                    for (int i=0;i<k;i++)
                        if ((bits>>i)&1u) subset.push_back(chosen[i]);
```

```
                    out.prefix[k-3].insert(encode(subset));
                }
                return;
            }
            int last=int(options.size())-k+int(chosen.size());
            for (int i=first;i<=last;i++) {
                chosen.push_back(options[i]);
                visit(i+1);
                chosen.pop_back();
            }
        };
        visit(0);
    }
    out.prefix[1].insert(out.prefix[0].begin(),out.prefix[0].end());
    require(out.counts[0]==298,"Triple catalogue mismatch");
    U64 expected=orientation==0 ? 21687 : 21823;
    require(out.counts[1]==expected,"Quadruple catalogue mismatch");
    return out;
}

// State describes only core rows and their previously available FULL values.
struct State {
    std::array<std::uint16_t,5> row{};
    std::array<U64,8> appeared{};
    std::uint8_t changed=0, returned=0, copied=0, extra=0;
    bool operator==(const State& b) const {
        return row==b.row && appeared==b.appeared &&
               changed==b.changed && returned==b.returned &&
               copied==b.copied && extra==b.extra;
    }
};
struct Hash {
    std::size_t operator()(const State& s) const {
        U64 h=14695981039346656O3ULL;
        auto add=[&](U64 x) { h^=x; h*=1099511628211ULL; h^=h>>29; };
        for (auto x:s.row) add(x);
        for (auto x:s.appeared) add(x);
        add(s.changed); add(s.returned); add(s.copied); add(s.extra);
        return std::size_t(h);
    }
};
bool has_full(const State& s,int x) {
    return (s.appeared[x/64]>>(x%64)) & U64(1);
}
void remember(State& s,int x) {
    s.appeared[x/64] |= U64(1)<<(x%64);
}
Mask projected(const State& s) {
    Mask m{};
    for (int i=0;i<8;i++) m[i%2] |= s.appeared[i];
    return m;
}
std::vector<int> xor_cost(const State& s,int dimension) {
    std::vector<int> cost(1<<dimension,999);
    cost[0]=0;
    for (int v=1;v<(1<<dimension);v++) if (has_full(s,v)) {
        // Exact zero/one update: pairs {x,x xor v} are disjoint.
        for (int x=0;x<(1<<dimension);x++) if (x<(x^v)) {
            int a=cost[x], b=cost[x^v];
            cost[x]=std::min(a,b+1);
            cost[x^v]=std::min(b,a+1);
        }
    }
    return cost;
}
struct Case {
    int orientation=0, family=0;
    std::vector<int> source, relay;
};
std::vector<Case> make_cases() {
    std::vector<Case> all;
    for (int a=0;a<6;a++) for (int b=a+1;b<6;b++)
        for (int c=0;c<6;c++) for (int d=c+1;d<6;d++)
```

```
            all.push_back({0,0,{a,b},{c,d}});
    for (int a=0;a<6;a++) for (int b=a+1;b<6;b++)
        for (int c=0;c<6;c++) all.push_back({1,1,{a,b},{c}});
    for (int a=0;a<6;a++) for (int b=0;b<6;b++)
        all.push_back({0,2,{a},{b}});
    for (int o=0;o<2;o++) for (int a=0;a<6;a++)
        all.push_back({o,3,{a},{}});
    require(all.size()==363,"Case cover size mismatch");
    return all;
}
struct Result {
    std::vector<U64> layers;
    U64 terminals=0, total=0;
    int best=999;
};
Result compute(const Case& c,const Catalogues& catalog) {
    int ns=int(c.source.size()), ny=int(c.relay.size());
    int nc=ns+1+ny, dimension=7+ny, maxdepth=12+ny;
    int slack=2-ns;
    std::vector<int> fixed, want, passive;
    for (int i=0;i<6;i++)
        if (std::find(c.source.begin(),c.source.end(),i)==c.source.end())
            fixed.push_back(1<<i);
    for (int i:c.source) want.push_back(1<<i);
    want.push_back(F[c.orientation][6]);
    for (int j=0;j<ny;j++)
        want.push_back((1<<(7+j)) ^ F[c.orientation][c.relay[j]]);
    for (int j=0;j<6;j++)
        if (std::find(c.relay.begin(),c.relay.end(),j)==c.relay.end())
            passive.push_back(F[c.orientation][j]);
    Mask base{}, output{};
    insert(base,0);
    for (int i=0;i<7;i++) insert(base,1<<i);
    for (int f:F[c.orientation]) insert(output,f);
    State initial;
    for (int i=0;i<ns;i++) initial.row[i]=1<<c.source[i];
    initial.row[ns]=64;
    for (int j=0;j<ny;j++) initial.row[ns+1+j]=1<<(7+j);
    for (int i=0;i<dimension;i++) remember(initial,1<<i);
    std::unordered_set<State,Hash> current, next;
    current.insert(initial);
    Result result;
    for (int depth=0;depth<=maxdepth;depth++) {
        result.layers.push_back(current.size());
        result.total+=current.size();
        for (const State& s:current) {
            if (s.changed!=((1<<ns)-1)) continue;
            bool terminal=true;
            for (int i=0;i<nc;i++) if (s.row[i]!=want[i]) terminal=false;
            if (!terminal) continue;
            result.terminals++;
            auto cost=xor_cost(s,dimension);
            int length=depth;
            for (int f:passive) length+=cost[f];
            result.best=std::min(result.best,length);
        }
        if (depth==maxdepth || current.empty()) break;
        next.clear();
        next.reserve(current.size()*2+1);
        for (const State& s:current) {
            Mask old=projected(s);
            std::vector<int> aux;
            for (int p=1;p<128;p++)
                if (member(old,p)&&!member(base,p)&&!member(output,p))
                    aux.push_back(p);
            for (int t=0;t<nc;t++)
            for (int j=0;j<nc+int(fixed.size());j++) {
                if (j==t) continue;
                int control=j<nc ? s.row[j] : fixed[j-nc];
                int value=s.row[t]^control, p=value&127;
                if (value==0) continue;
                State nextstate=s;
                bool mandatory=false;
```

```
                if (t<ns) {
                    nextstate.changed|=1<<t;
                    if (value==(1<<c.source[t]) &&
                        ((s.changed>>t)&1) && !((s.returned>>t)&1)) {
                        nextstate.returned|=1<<t;
                        mandatory=true;
                    }
                }
                if (t>ns && (s.row[t]&127)==0 && p!=0 &&
                    !((s.copied>>(t-ns-1))&1)) {
                    nextstate.copied|=1<<(t-ns-1);
                    mandatory=true;
                }
                bool seen=member(base,p)||member(old,p);
                if (seen&&!mandatory) nextstate.extra++;
                if (nextstate.extra>slack) continue;
                auto enlarged=aux;
                if (!seen&&!member(output,p)) enlarged.push_back(p);
                int capacity=5-ns-nextstate.extra;
                if (int(enlarged.size())>capacity) continue;
                const auto& allowed=catalog.prefix[capacity<=3 ? 0 : 1];
                if (!allowed.count(encode(enlarged))) continue;
                nextstate.row[t]=value;
                remember(nextstate,value);
                if (slack==0) {
                    Mask known=projected(nextstate);
                    bool valid=true;
                    for (int i=ns;i<nc && valid;i++) {
                        int f=want[i]&127;
                        if (!member(known,f)||nextstate.row[i]==want[i])
                            continue;
                        if (i==ns || (nextstate.row[i]&127)!=0) {
                            valid=false;
                            break;
                        }
                        bool clean=false;
                        for (int j2=0;j2<nc;j2++)
                            if (nextstate.row[j2]==f) clean=true;
                        if (!clean) valid=false;
                    }
                    for (int f:passive)
                        if (member(known,f)&&!has_full(nextstate,f))
                            valid=false;
                    if (!valid) continue;
                }
                int different=0, untouched=0;
                std::set<int> missing;
                for (int i=0;i<nc;i++) if (nextstate.row[i]!=want[i]) {
                    different++;
                    int increment=nextstate.row[i]^want[i];
                    bool available=false;
                    for (int j2=0;j2<nc;j2++)
                        if (nextstate.row[j2]==increment ||
                            want[j2]==increment) available=true;
                    for (int f:fixed) if (f==increment) available=true;
                    if (!available) missing.insert(increment);
                }
                for (int i=0;i<ns;i++)
                    if (!((nextstate.changed>>i)&1)) untouched++;
                int bound=std::max(different+2*untouched,
                           different+(int(missing.size())+1)/2);
                if (depth+1+bound>maxdepth) continue;
                next.insert(nextstate);
            }
        }
        current.swap(next);
    }
    return result;
}

void foundations() {
    std::vector<int> span{0};
    for (int f:{73,82,100}) {
```

```
        auto copy=span;
        for (int x:copy) span.push_back(x^f);
    }
    std::sort(span.begin(),span.end());
    require(std::set<int>(span.begin(),span.end()).size()==8,"Rank of H");
    std::set<int> start, forms, cosets;
    for (int i=0;i<7;i++) start.insert(1<<i);
    forms=start;
    forms.insert(F[0].begin(),F[0].end());
    int outside=0;
    for (int x:forms) {
        if (std::binary_search(span.begin(),span.end(),x)) continue;
        outside++;
        int representative=127;
        for (int h:span) representative=std::min(representative,x^h);
        require(representative!=0,"Zero coset");
        cosets.insert(representative);
    }
    for (int x:start)
        require(!std::binary_search(span.begin(),span.end(),x),"Input in H");
    require(outside==9&&cosets.size()==9,"Coset condition");
    const std::array<std::array<int,2>,10> xor_program = {{
        {1,8},{2,16},{4,32},{9,64},{18,64},
        {36,64},{1,2},{2,4},{9,18},{18,36}
    }};
    auto available=start;
    for (auto op:xor_program) {
        require(available.count(op[0])&&available.count(op[1]),"XOR input");
        require(!available.count(op[0]^op[1]),"XOR repeat");
        available.insert(op[0]^op[1]);
    }
    for (int f:F[0]) require(available.count(f),"XOR output");
    for (int j=0;j<7;j++) {
        int column=0;
        for (int i=0;i<7;i++) column|=((F[0][i]>>j)&1)<<i;
        require(column==F[1][j],"Transposition mismatch");
    }
    for (int u=0;u<=2;u++) for (int v=0;v<=2;v++)
        require(16+v+std::min(4,6-u-v)>=20,"Zero-sided bound");
    const std::array<std::array<int,2>,19> witness = {{
        {0,9},{1,9},{1,10},{2,10},{2,12},{5,12},
        {3,0},{0,11},{4,1},{1,11},{1,12},{1,7},{0,6},
        {8,7},{8,6},{2,8},{5,8},{4,1},{3,0}
    }};
    std::array<int,13> rows{};
    for (int i=0;i<13;i++) rows[i]=1<<i;
    for (auto g:witness) rows[g[1]]^=rows[g[0]];
    require(rows==TARGET,"19-gate witness mismatch");
    // Check that both normalized 13-row tasks are the stated full endpoints.
    std::array<std::array<int,13>,2> order = {{
        {0,1,2,3,4,5,8,6,7,9,10,11,12},
        {6,7,9,10,11,12,8,0,1,2,3,4,5}
    }};
    for (int o=0;o<2;o++) for (int i=0;i<13;i++) {
        int rowmask=0;
        for (int j=0;j<13;j++) {
            int bit=o==0 ? (TARGET[order[o][i]]>>order[o][j])&1
                         : (TARGET[order[o][j]]>>order[o][i])&1;
            rowmask|=bit<<j;
        }
        int expected=i<6 ? 1<<i : (i==6 ? F[o][6]
                                        : (1<<i)^F[o][i-7]);
        require(rowmask==expected,"Normalized full matrix mismatch");
    }
}
std::mutex output_mutex;
void print_case(int id,const Case& c,const Result& r) {
    std::lock_guard<std::mutex> guard(output_mutex);
    std::cout<<"CASE {\"id\":"<<id<<",\"orientation\":"<<c.orientation
             <<",\"family\":"<<c.family<<",\"layers\":[";
    for (std::size_t i=0;i<r.layers.size();i++) {
        if (i) std::cout<<",";
        std::cout<<r.layers[i];
```

```
    }
    std::cout<<"],\"total\":"<<r.total<<",\"terminals\":"<<r.terminals
             <<",\"best\":"<<r.best<<",\"status\":\""
             <<(r.best>18?"zero_support":"positive")<<"\"}"<<std::endl;
}
int main(int argc,char** argv) {
    try {
        foundations();
        std::array<Catalogues,2> catalog =
            {generate_catalogues(0),generate_catalogues(1)};
        std::cout<<"CATALOGUES 298 21687 298 21823\nFOUNDATIONS_OK\n";
        auto cases=make_cases();
        if (argc==3&&std::string(argv[1])=="--one") {
            int id=std::stoi(argv[2]);
            require(id>=0&&id<int(cases.size()),"Invalid case index");
            auto r=compute(cases[id],catalog[cases[id].orientation]);
            print_case(id,cases[id],r);
            std::cout<<"PARTIAL_CHECK_ONLY\n";
            return r.best>18?0:1;
        }
        require(argc<=2,"Usage: hgp13_verify [jobs], or --one index");
        int jobs=argc==2?std::stoi(argv[1]):1;
        require(jobs>=1&&jobs<=8,"jobs must be 1 to 8");
        std::vector<Result> results(cases.size());
        std::atomic<int> cursor{0};
        std::atomic<bool> failed{false};
        std::exception_ptr error;
        std::mutex error_mutex;
        std::vector<std::thread> pool;
        for (int j=0;j<jobs;j++) pool.emplace_back([&]() {
            try {
                while (!failed.load()) {
                    int id=cursor.fetch_add(1);
                    if (id>=int(cases.size())) break;
                    results[id]=compute(cases[id],catalog[cases[id].orientation]);
                    print_case(id,cases[id],results[id]);
                }
            } catch (...) {
                std::lock_guard<std::mutex> guard(error_mutex);
                if (!error) error=std::current_exception();
                failed.store(true);
            }
        });
        for (auto& thread:pool) thread.join();
        if (error) std::rethrow_exception(error);
        std::array<U64,4> numbers{}, totals{}, terminals{}, maxima{};
        U64 total=0;
        for (std::size_t id=0;id<cases.size();id++) {
            const auto& r=results[id];
            require(!r.layers.empty(),"An uncomputed case remains");
            require(r.best>18,"A candidate of length <=18 remains");
            int f=cases[id].family;
            numbers[f]++; totals[f]+=r.total; terminals[f]+=r.terminals;
            maxima[f]=std::max(maxima[f],*std::max_element(r.layers.begin(),r.layers.end()));
            total+=r.total;
        }
        for (int f=0;f<4;f++)
            std::cout<<"FAMILY "<<f<<" CASES "<<numbers[f]
                     <<" LABELS "<<totals[f]<<" MAX_LAYER "<<maxima[f]
                     <<" TERMINALS "<<terminals[f]<<"\n";
        std::cout<<"COMPLETED "<<cases.size()<<" TOTAL_LABELS "<<total<<"\n"
                 <<"VERIFIED_FIXED_MATRIX_MINIMUM_19\n";
        return 0;
    } catch (const std::exception& e) {
        std::cerr<<"VERIFICATION_FAILED: "<<e.what()<<"\n";
        return 1;
    }
}
```